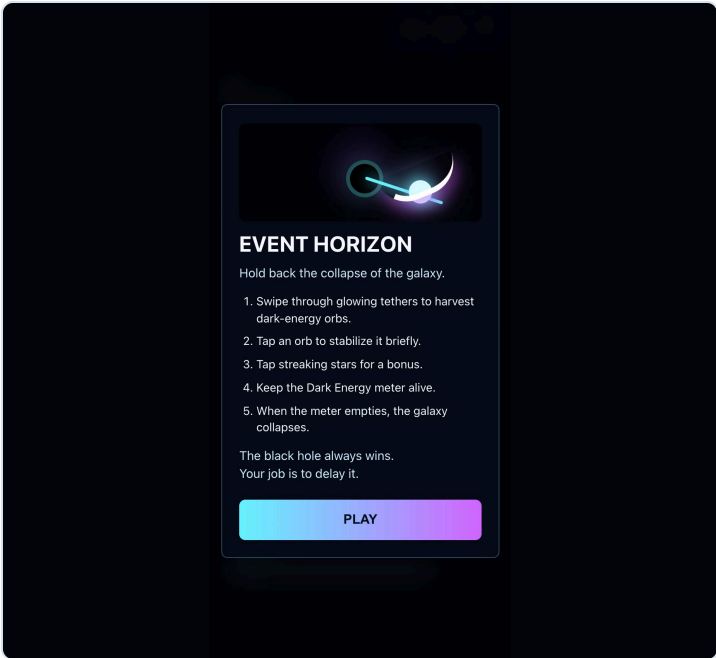


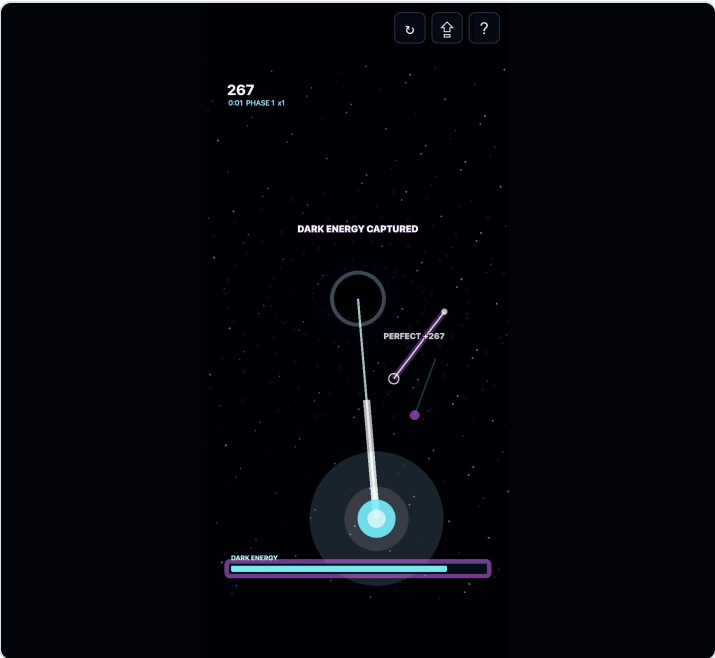
Event Horizon Iteration 02 Report

Mobile input, clarity, onboarding, and game feel pass.

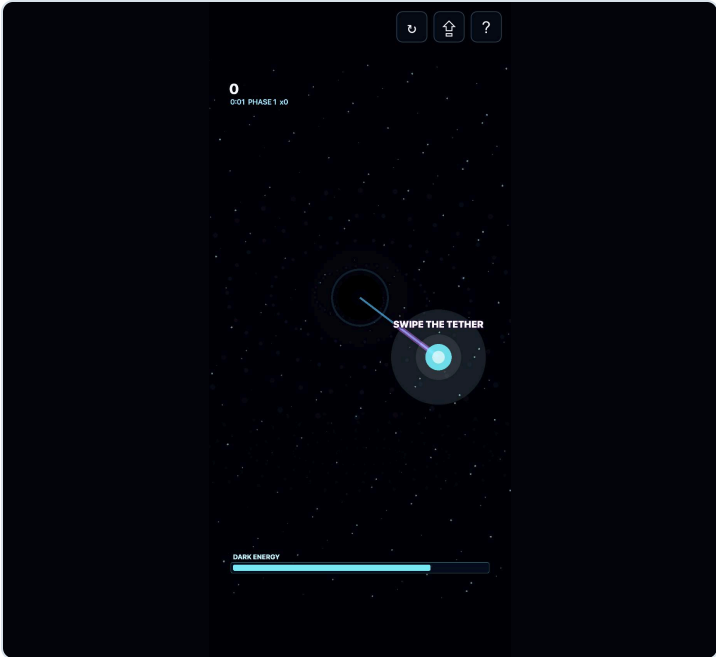
Screenshots



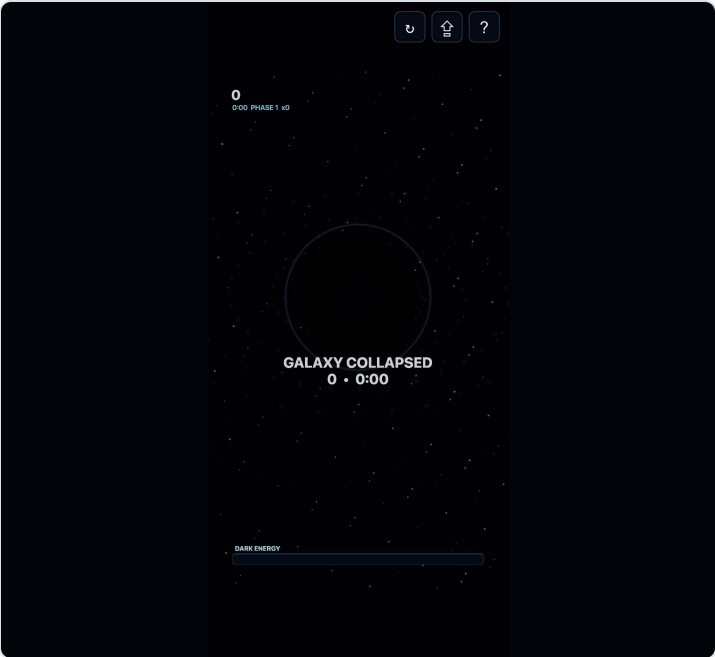
Help overlay



Swipe trail and harvest feedback



Tutorial gameplay



Collapse state

Event Horizon Iteration 02 Report

Summary

Iteration 02 focuses on mobile input reliability, player comprehension, and game feel. Swipes now capture full gesture paths, render the exact interpreted trail, and hit-test against the glowing tether near an orb instead of requiring precision on the orb body. A first-run help overlay, a slower onboarding ramp, clearer Dark Energy HUD labels, hit/miss feedback, and debug hooks make smartphone testing much easier.

Diagnosis Of The Smartphone Swipe Issue

The first playable treated a swipe mostly as a start/end segment and classified it using world-space movement. On a phone, that made the gesture brittle because small browser-coordinate movements can become large or inconsistent after viewport scaling, and the player naturally swipes through the visible tether rather than through a small orb. There was also no rendered trail or result message, so a player could not tell whether the game saw a tap, swipe, miss, or harvest.

What Changed

- Pointer Events now capture full gesture paths, use CSS-pixel thresholds, use coalesced samples when available, and include TouchEvent fallback.
- CSS locks the gameplay surface against scroll, zoom, selection, overscroll, and long-press callouts.
- Swipe simulation now quantizes and records path samples, then tests each gesture segment against active tether segments.
- The first 30 seconds now use slower, larger, brighter tutorial orbs with delayed flyby/arm pressure.
- Pixi renders live/fading trails, start dots, end sparks, hit/miss labels, orb/tether flashes, Dark Energy meter glow, and energy pulses.
- HTML help opens automatically on first visit, pauses the simulation, and can be reopened with the ? button.
- Debug hooks expose snapshots, replay payloads, last gesture details, and world-space tap/swipe simulation.

Exact Files Changed

- README.md
- index.html
- package.json
- scripts/capture-iteration-02-artifacts.mjs
- scripts/generate-iteration-02-report.mjs
- src/game/EventHorizonGame.ts
- src/game/EventHandler.ts
- src/game/Simulation.ts
- src/game/gestures.ts
- src/game/types.ts
- src/main.ts
- src/styles.css
- tests/e2e/playable.spec.ts
- tests/simulation.test.ts
- docs/artifacts/iteration-02-collapse-mobile.jpg
- docs/artifacts/iteration-02-gameplay-mobile.jpg
- docs/artifacts/iteration-02-help-mobile.jpg
- docs/artifacts/iteration-02-swipe-trail-mobile.jpg
- docs/artifacts/iteration-02-test-results.txt

Binary Or Generated Artifacts

- docs/artifacts/iteration-02-collapse-mobile.jpg
- docs/artifacts/iteration-02-gameplay-mobile.jpg
- docs/artifacts/iteration-02-help-mobile.jpg
- docs/artifacts/iteration-02-swipe-trail-mobile.jpg
- docs/artifacts/iteration-02-test-results.txt

Test Results

```
Event Horizon iteration 02 verification
Date: 2026-05-30T02:21:14Z

$ npm run build
passed: TypeScript noEmit and Vite production build completed.

$ npm run lint
passed: ESLint completed with no findings.

$ npm run test
passed: 2 test files, 9 unit tests.
- score-submit.test.ts: 2 passed
- simulation.test.ts: 7 passed, including tether hit detection, miss behavior, path simplification, deterministic replay, and legacy simple swipe.

$ npm run test:e2e
passed: 6 mobile Chrome Playwright tests.
- help overlay opens on first visit and closes with play
- smoke renders playable Pixi canvas
- tap and tether swipe are captured into replay payload
- posterizer exports a compact png data url
- miss swipe records visible feedback state
- end-state collapse is reachable

$ npm run score:test
passed: status 200, ok true, score 1280, survivalMs 67421.

$ npm run capture:iteration-02
passed: captured iteration-02-help-mobile.jpg, iteration-02-gameplay-mobile.jpg, iteration-02-swipe-trail-mobile.jpg, iteration-02-collapse-mobile.jpg.

Manual physical phone test
not run in this Codex session; Playwright mobile simulation passed.
```

Screenshots

- docs/artifacts/iteration-02-help-mobile.jpg
- docs/artifacts/iteration-02-swipe-trail-mobile.jpg
- docs/artifacts/iteration-02-gameplay-mobile.jpg
- docs/artifacts/iteration-02-collapse-mobile.jpg

Known Limitations

- Playwright mobile simulation passed; this has not been manually verified on a physical smartphone in this run.
- Score submission still uses the minimal validation endpoint; GitHub Pages does not host Netlify Functions.
- Tutorial state is intentionally simple and localStorage-based, not a full scripted tutorial engine.
- Touch fallback is present, but the primary tested path is modern Chrome Pointer Events.

Next Recommended Iteration

- Test on an actual iPhone and Android device, then tune CSS-pixel thresholds and tether width from observations.
- Add a replay playback viewer so path-based replay data can be reviewed visually.
- Add sound and haptic settings, plus a reduced-motion option.
- Tune scoring, streak rewards, and phase transitions once the core gesture feels good.

Full Source Code For Changed Text Files

README.md

```
# Event Horizon

Event Horizon is a mobile-first, one-thumb casual survival prototype about delaying a galaxy's collapse into a black hole. The first vertical slice is a plain Vite + PixiJS v8 site with deterministic simulation state kept outside the renderer.

## Current Slice

- Logical portrait playfield: `1080 x 1920`
- PixiJS canvas renderer with WebGL preference
- Fixed `60 Hz` simulation step decoupled from render frames
- Seeded `mulberry32` RNG and replay payloads from seed + input timings
- Path-based tap/swipe input recording with mobile Pointer Events and TouchEvent fallback
- Dark-energy orbs, tether-based swipe capture, flyby bonuses, and shadow-arm hazards
- First-run help overlay, tutorial ramp, visible swipe trails, hit/miss feedback, and debug hooks
- Phase-based black-hole visibility and gravity pressure
- Bottom dark-energy meter, score/time HUD, and collapse end animation
- Share poster export from three captured gameplay frames
- Minimal Netlify score submit function plus Google Apps Script sample

## Assumptions

- The initial repo was empty, so this is a greenfield Vite scaffold.
- Custom deterministic motion is enough for the vertical slice; no Matter.js or Planck.js dependency is needed yet.
- GitHub Pages is the public static target and uses `/event-horizon/` as the base path.
- Netlify deploys should set `VITE_BASE_PATH=/` through the included `netlify.toml`.
- The score function validates and acknowledges payloads only; durable storage is a backlog item.

## Commands

```bash
npm install
npm run dev
npm run build
npm run lint
npm run test
npm run test:e2e
npm run score:test
npm run capture:iteration-02
npm run report:iteration-02
npm run report:pdf
```

Local Vite serves the game at:

```text
http://127.0.0.1:5173/event-horizon/
```

## Deployment

### GitHub Pages

The default Vite base path is `/event-horizon/`, so `npm run build` produces static assets compatible with:

```text
https://kevinhegg.github.io/event-horizon/
```

### Netlify

`netlify.toml` uses:

```toml
[build]
 command = "VITE_BASE_PATH=/ npm run build"
 publish = "dist"

[functions]
 directory = "netlify/functions"
```

The score endpoint is available at:

```text
./.netlify/functions/score-submit
```

For local Netlify function routing, run Netlify CLI from the repo root:
```

```

\\\`bash
npx netlify dev
\\\`

## Replay Payload Shape

\\\`json
{
  "version": 1,
  "seed": "eh-2026-05-29-alpha",
  "startedAt": 1780051200000,
  "survivalMs": 67421,
  "score": 1280,
  "energyCaptured": 87,
  "maxStreak": 24,
  "tapEvents": [],
  "swipeEvents": [],
  "phaseTransitions": []
}
\\\`

## First PR Workflow

\\\`bash
git switch -c feat/first-playable
git add .
git commit -m "Build first playable Event Horizon slice"
git push -u origin feat/first-playable
gh pr create --base main --head feat/first-playable --title "Build first playable Event Horizon slice" --body-file docs/pr-body.md
\\\`

```

index.html

```

<!doctype html>
<html lang="en-US">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover" />
  <meta name="theme-color" content="#03040a" />
  <meta name="description" content="Event Horizon, a one-thumb galaxy survival prototype." />
  <title>Event Horizon</title>
</head>
<body>
  <main id="app" aria-label="Event Horizon game">
    <div id="game-shell">
      <div id="game-root" aria-label="Playable canvas"></div>
      <div id="status-layer" aria-live="polite">
        <button id="restart-button" type="button" aria-label="Restart run" title="Restart run"></button>
        <button id="share-button" type="button" aria-label="Create share poster" title="Create share poster"></button>
        <button id="help-button" type="button" aria-label="How to play" title="How to play"></button>
        <a id="poster-link" download="event-horizon-poster.png" aria-label="Download share poster"></a>
      </div>
      <section id="help-overlay" aria-modal="true" role="dialog" aria-labelledby="help-title" hidden>
        <div id="help-panel">
          <div class="help-example" aria-hidden="true">
            <span class="help-hole"></span>
            <span class="help-tether"></span>
            <span class="help-orb"></span>
            <span class="help-swipe"></span>
          </div>
          <h1 id="help-title">EVENT HORIZON</h1>
          <p>Hold back the collapse of the galaxy.</p>
          <ol>
            <li>Swipe through glowing tethers to harvest dark-energy orbs.</li>
            <li>Tap an orb to stabilize it briefly.</li>
            <li>Tap streaking stars for a bonus.</li>
            <li>Keep the Dark Energy meter alive.</li>
            <li>When the meter empties, the galaxy collapses.</li>
          </ol>
          <p>The black hole always wins.<br />Your job is to delay it.</p>
          <button id="help-play-button" type="button">PLAY</button>
        </div>
      </section>
    </div>
  </main>
  <script type="module" src="/src/main.ts"></script>
</body>
</html>

```

package.json

```

{
  "name": "event-horizon",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview --host 127.0.0.1",
    "test": "vitest run",
    "test:e2e": "playwright test",
    "lint": "eslint .",
    "score:test": "node scripts/test-score-submit.mjs",
    "capture:artifacts": "node scripts/capture-artifacts.mjs",
    "capture:iteration-02": "node scripts/capture-iteration-02-artifacts.mjs",
    "report:pdf": "node scripts/generate-report-pdf.mjs",
    "report:iteration-02": "node scripts/generate-iteration-02-report.mjs"
  },
  "dependencies": {
    "pixi.js": "8.18.1"
  },
  "devDependencies": {
    "@eslint/js": "9.39.1",
    "@playwright/test": "1.57.0",
    "@types/node": "24.12.4",
    "eslint": "9.39.1",
    "typescript": "5.9.3",
    "typescript-eslint": "8.49.0",
    "vite": "8.0.14",
    "vitest": "4.0.15"
  }
}

```

```
}  
}
```

scripts/capture-iteration-02-artifacts.mjs

```
import { mkdir } from 'node:fs/promises';  
import { chromium } from '@playwright/test';  
  
const WORLD_WIDTH = 1080;  
const WORLD_HEIGHT = 1920;  
const BLACK_HOLE_X = WORLD_WIDTH / 2;  
const BLACK_HOLE_Y = WORLD_HEIGHT * 0.44;  
  
const outputDir = new URL('../docs/artifacts/', import.meta.url);  
await mkdir(outputDir, { recursive: true });  
  
const browser = await chromium.launch({ channel: 'chrome' });  
const page = await browser.newPage({  
  viewport: { width: 390, height: 844 },  
  deviceScaleFactor: 2,  
  isMobile: true,  
  hasTouch: true  
});  
  
await page.addInitScript(() => window.localStorage.clear());  
await page.goto('http://127.0.0.1:5173/event-horizon/', { waitUntil: 'networkidle' });  
await page.locator('canvas').waitFor({ state: 'visible', timeout: 10000 });  
await page.screenshot({  
  path: new URL('iteration-02-help-mobile.jpg', outputDir).pathname,  
  type: 'jpeg',  
  quality: 72,  
  fullPage: false  
});  
  
await page.locator('#help-play-button').click();  
await page.waitForFunction(() => {  
  const snapshot = window.__EVENT_HORIZON__?.getSnapshot();  
  return snapshot?.orbs.some((orb) => orb.active && orb.tutorial && !orb.captured);  
});  
await page.waitForTimeout(180);  
await page.screenshot({  
  path: new URL('iteration-02-gameplay-mobile.jpg', outputDir).pathname,  
  type: 'jpeg',  
  quality: 70,  
  fullPage: false  
});  
  
const swipe = await makeTutorialTetherSwipe(page);  
await page.mouse.move(swipe.start.x, swipe.start.y);  
await page.mouse.down();  
await page.mouse.move(swipe.mid.x, swipe.mid.y, { steps: 4 });  
await page.mouse.move(swipe.end.x, swipe.end.y, { steps: 5 });  
await page.mouse.up();  
await page.waitForTimeout(130);  
await page.screenshot({  
  path: new URL('iteration-02-swipe-trail-mobile.jpg', outputDir).pathname,  
  type: 'jpeg',  
  quality: 76,  
  fullPage: false  
});  
  
await page.evaluate(() =>  
  window.__EVENT_HORIZON_DEBUG__?.simulateSwipeWorld([  
    { x: 80, y: 1760 },  
    { x: 180, y: 1840 }  
  ]) );  
await page.waitForTimeout(250);  
  
await page.evaluate(() => window.__EVENT_HORIZON__?.restart());  
await page.waitForTimeout(150);  
await page.evaluate(() => window.__EVENT_HORIZON__?.forceEnd());  
await page.waitForTimeout(1100);  
await page.screenshot({  
  path: new URL('iteration-02-collapse-mobile.jpg', outputDir).pathname,  
  type: 'jpeg',  
  quality: 66,  
  fullPage: false  
});  
  
await browser.close();  
  
console.log('Captured docs/artifacts/iteration-02-help-mobile.jpg');  
console.log('Captured docs/artifacts/iteration-02-gameplay-mobile.jpg');  
console.log('Captured docs/artifacts/iteration-02-swipe-trail-mobile.jpg');  
console.log('Captured docs/artifacts/iteration-02-collapse-mobile.jpg');  
  
async function makeTutorialTetherSwipe(page) {  
  const box = await page.locator('canvas').boundingBox();  
  if (!box) {  
    throw new Error('Canvas box unavailable.');  }  
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());  
  const orb = snapshot?.orbs.find((candidate) => candidate.active && candidate.tutorial && !candidate.captured);  
  if (!orb) {  
    throw new Error('Tutorial orb unavailable.');  }  
  const targetX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * 0.78;  
  const targetY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * 0.78;  
  const dx = orb.x - BLACK_HOLE_X;  
  const dy = orb.y - BLACK_HOLE_Y;  
  const length = Math.max(1, Math.hypot(dx, dy));  
  const normalX = -dy / length;  
  const normalY = dx / length;  
  return {  
    start: worldToScreen(box, targetX - normalX * 150, targetY - normalY * 150),  
    mid: worldToScreen(box, targetX, targetY),  
    end: worldToScreen(box, targetX + normalX * 150, targetY + normalY * 150)  
  };  
}  
  
function worldToScreen(box, x, y) {  
  const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);  
  const offsetX = (box.width - WORLD_WIDTH * scale) / 2;  
  const offsetY = (box.height - WORLD_HEIGHT * scale) / 2;  
  return {  
    x: x * scale + offsetX,  
    y: y * scale + offsetY  
  };  
}
```

```

    x: box.x + offsetX + x * scale,
    y: box.y + offsetY + y * scale
  });
}

```

scripts/generate-iteration-02-report.mjs

```

import { execFileSync } from 'node:child_process';
import { existsSync } from 'node:fs';
import { mkdir, readFile, stat, writeFile } from 'node:fs/promises';
import { extname } from 'node:path';
import { chromium } from '@playwright/test';

const repoRoot = new URL('../', import.meta.url);
const docsDir = new URL('../docs/', import.meta.url);
const artifactsDir = new URL('artifacts/', docsDir);
const reportPath = new URL('iteration-02-report.md', docsDir);
const pdfPath = new URL('iteration-02-report.pdf', docsDir);
const testResultsPath = new URL('artifacts/iteration-02-test-results.txt', docsDir);
const baseRef = process.env.REPORT_BASE_REF ?? 'origin/main';

await mkdir(docsDir, { recursive: true });

const changedFiles = await collectChangedFiles();
const sourceFiles = changedFiles.filter((file) => isTextSource(file));
const binaryFiles = changedFiles.filter((file) => !isTextSource(file));
const testResults = existsSync(testResultsPath)
  ? await readFile(testResultsPath, 'utf8')
  : 'Test result log was not present when this report was generated.';

const markdown = await buildMarkdown(sourceFiles, binaryFiles, testResults);
await writeFile(reportPath, markdown);

const html = await buildHtml(markdown);
const browser = await chromium.launch({ channel: 'chrome' });
const page = await browser.newPage();
await page.setContent(html, { waitUntil: 'load' });
await page.pdf({
  path: pdfPath.pathname,
  format: 'Letter',
  printBackground: true,
  margin: {
    top: '0.5in',
    right: '0.46in',
    bottom: '0.5in',
    left: '0.46in'
  }
});
await browser.close();

const size = await stat(pdfPath);
console.log(`Wrote docs/iteration-02-report.md`);
console.log(`Wrote docs/iteration-02-report.pdf (${Math.round(size.size / 1024)} KiB)`);

async function collectChangedFiles() {
  const names = new Set([
    ...lines(git(['diff', `${baseRef}...HEAD`, '--name-only'])),
    ...lines(git(['diff', '--cached', '--name-only'])),
    ...lines(git(['diff', '--name-only'])),
    ...lines(git(['ls-files', '--others', '--exclude-standard']))
  ]);
  names.delete('docs/iteration-02-report.md');
  names.delete('docs/iteration-02-report.pdf');
  return [...names].sort();
}

async function buildMarkdown(sourceFiles, binaryFiles, testResults) {
  const sourceBlocks = [];
  for (const file of sourceFiles) {
    const content = await readFile(new URL(file, repoRoot), 'utf8');
    sourceBlocks.push(`### ${file}\n\n\`${languageFor(file)}\`${content.replaceAll('\`', '\\`')}\n\n\`${languageFor(file)}\`${content.replaceAll('\`', '\\`')}`);
  }

  return `# Event Horizon Iteration 02 Report

## Summary

Iteration 02 focuses on mobile input reliability, player comprehension, and game feel. Swipes now capture full gesture paths, render the exact interpreted trail, and hit-test against the glowing tether near an orb instead of requiring precision on the orb body. A first-run help overlay, a slower onboarding ramp, clearer Dark Energy HUD labels, hit/miss feedback, and debug hooks make smartphone testing much easier.

## Diagnosis Of The Smartphone Swipe Issue

The first playable treated a swipe mostly as a start/end segment and classified it using world-space movement. On a phone, that made the gesture brittle because small browser-coordinate movements can become large or inconsistent after viewport scaling, and the player naturally swipes through the visible tether rather than through a small orb. There was also no rendered trail or result message, so a player could not tell whether the game saw a tap, swipe, miss, or harvest.

## What Changed

- Pointer Events now capture full gesture paths, use CSS-pixel thresholds, use coalesced samples when available, and include TouchEvent fallback.
- CSS locks the gameplay surface against scroll, zoom, selection, overscroll, and long-press callouts.
- Swipe simulation now quantizes and records path samples, then tests each gesture segment against active tether segments.
- The first 30 seconds now use slower, larger, brighter tutorial orbs with delayed flyby/arm pressure.
- Pixi renders live/fading trails, start dots, end sparks, hit/miss labels, orb/tether flashes, Dark Energy meter glow, and energy pulses.
- HTML help opens automatically on first visit, pauses the simulation, and can be reopened with the ? button.
- Debug hooks expose snapshots, replay payloads, last gesture details, and world-space tap/swipe simulation.

## Exact Files Changed

${[...sourceFiles, ...binaryFiles].map((file) => ` - ${file}`).join('\n')}

## Binary Or Generated Artifacts

${binaryFiles.length > 0 ? binaryFiles.map((file) => ` - ${file}`).join('\n') : ` - None`}

## Test Results

\`\`\`text
${testResults.trim()}
\`\`\`

## Screenshots

- docs/artifacts/iteration-02-help-mobile.jpg
- docs/artifacts/iteration-02-swipe-trail-mobile.jpg
- docs/artifacts/iteration-02-gameplay-mobile.jpg

```

```

- docs/artifacts/iteration-02-collapse-mobile.jpg

## Known Limitations

- Playwright mobile simulation passed; this has not been manually verified on a physical smartphone in this run.
- Score submission still uses the minimal validation endpoint; GitHub Pages does not host Netlify Functions.
- Tutorial state is intentionally simple and localStorage-based, not a full scripted tutorial engine.
- Touch fallback is present, but the primary tested path is modern Chrome Pointer Events.

## Next Recommended Iteration

- Test on an actual iPhone and Android device, then tune CSS-pixel thresholds and tether width from observations.
- Add a replay playback viewer so path-based replay data can be reviewed visually.
- Add sound and haptic settings, plus a reduced-motion option.
- Tune scoring, streak rewards, and phase transitions once the core gesture feels good.

## Full Source Code For Changed Text Files

${sourceBlocks.join('\n\n')}
`;
}

async function buildHtml(markdown) {
  const images = await Promise.all(
    [
      ['Help overlay', new URL('iteration-02-help-mobile.jpg', artifactsDir)],
      ['Swipe trail and harvest feedback', new URL('iteration-02-swipe-trail-mobile.jpg', artifactsDir)],
      ['Tutorial gameplay', new URL('iteration-02-gameplay-mobile.jpg', artifactsDir)],
      ['Collapse state', new URL('iteration-02-collapse-mobile.jpg', artifactsDir)]
    ].map(async ([label, url]) => {
      if (!existsSync(url)) {
        return '';
      }
      const bytes = await readFile(url);
      return `<figure><figcaption>${escapeHtml(label)}</figcaption></figure>`;
    })
  );
}

return `<!doctype html>
<html lang="en-US">
<head>
  <meta charset="utf-8" />
  <title>Event Horizon Iteration 02 Report</title>
  <style>
    @page { size: Letter; margin: 0.5in 0.46in; }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      color: #15202b;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
      font-size: 10.5px;
      line-height: 1.36;
    }
    h1, h2, h3 { color: #07111f; line-height: 1.12; margin: 0.56rem 0 0.28rem; break-after: avoid; }
    h1 { font-size: 22px; border-bottom: 2px solid #2f7eaf; padding-bottom: 7px; }
    h2 { font-size: 15px; }
    h3 { font-size: 11.5px; }
    p { margin: 0 0 0.42rem; }
    ul, ol { margin: 0 0 0.5rem 1.1rem; padding: 0; }
    li { margin: 0.08rem 0; }
    code { font-family: "SFMono-Regular", Menlo, Consolas, monospace; font-size: 8px; }
    pre {
      margin: 0.35rem 0 0.65rem;
      padding: 8px;
      white-space: pre-wrap;
      overflow-wrap: anywhere;
      background: #fff3f6f9;
      border: 1px solid #d9e2ea;
      border-radius: 5px;
      font-family: "SFMono-Regular", Menlo, Consolas, monospace;
      font-size: 6.6px;
      line-height: 1.22;
    }
    .cover {
      background: #07111f;
      color: #ee99ff;
      border-radius: 8px;
      padding: 18px;
      margin-bottom: 12px;
    }
    .cover h1 { color: #fff; border-color: #80e3ff; margin-top: 0; }
    .cover p { color: #cbeaf4; margin-bottom: 0; }
    .gallery {
      display: grid;
      grid-template-columns: repeat(2, 1fr);
      gap: 10px;
      margin: 8px 0 12px;
      break-inside: avoid;
    }
    figure { margin: 0; break-inside: avoid; }
    img {
      display: block;
      width: 100%;
      max-height: 330px;
      object-fit: contain;
      border: 1px solid #d9e2ea;
      border-radius: 6px;
      background: #03040a;
    }
    figcaption { margin-top: 4px; color: #526171; font-size: 9px; text-align: center; }
  </style>
</head>
<body>
  <section class="cover">
    <h1>Event Horizon Iteration 02 Report</h1>
    <p>Mobile input, clarity, onboarding, and game feel pass.</p>
  </section>
  <section>
    <h2>Screenshots</h2>
    <div class="gallery">${images.join('')}</div>
  </section>
  ${markdownToHtml(markdown)}
</body>
</html>`;
}

function markdownToHtml(markdown) {
  const lines = markdown.split('\n');
  const html = [];

```

```

let inCode = false;
let codeLines = [];
let listOpen = false;

for (const line of lines) {
  if (line.startsWith('`')) {
    if (inCode) {
      html.push('<pre>${escapeHtml(codeLines.join('\n'))}</pre>');
      codeLines = [];
    }
    inCode = !inCode;
    continue;
  }

  if (inCode) {
    codeLines.push(line);
    continue;
  }

  if (line.startsWith('- ')) {
    if (!listOpen) {
      html.push('<ul>');
      listOpen = true;
    }
    html.push('<li>${inlineMarkdown(line.slice(2))}</li>');
    continue;
  }

  if (listOpen) {
    html.push('</ul>');
    listOpen = false;
  }

  if (line.startsWith('# ')) {
    html.push('<h1>${inlineMarkdown(line.slice(2))}</h1>');
  } else if (line.startsWith('## ')) {
    html.push('<h2>${inlineMarkdown(line.slice(3))}</h2>');
  } else if (line.startsWith('### ')) {
    html.push('<h3>${inlineMarkdown(line.slice(4))}</h3>');
  } else if (/^\d+\.\s/.test(line)) {
    html.push('<p>${inlineMarkdown(line)}</p>');
  } else if (line.trim()) {
    html.push('<p>${inlineMarkdown(line)}</p>');
  }
}

if (listOpen) {
  html.push('</ul>');
}
return html.join('\n');
}

function inlineMarkdown(value) {
  return escapeHtml(value)
    .replace(/`([^\`]+)`/g, '<code>$1</code>')
    .replace(/\[([^\]]+)\]\([([^\]]+)\)/g, '<a href="$2">$1</a>');
}

function escapeHtml(value) {
  return String(value)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}

function git(args) {
  try {
    return execFileSync('git', args, {
      cwd: repoRoot,
      encoding: 'utf8',
      maxBuffer: 1024 * 1024 * 16
    });
  } catch {
    return '';
  }
}

function lines(value) {
  return value
    .split('\n')
    .map((line) => line.trim())
    .filter(Boolean);
}

function isTextSource(file) {
  if (file.startsWith('docs/artifacts/') || file.endsWith('.pdf')) {
    return false;
  }
  return ['.css', '.html', '.js', '.json', '.md', '.mjs', '.ts', '.tsx', '.yaml'].includes(extname(file));
}

function languageFor(file) {
  const extension = extname(file);
  if (extension === '.ts') {
    return 'ts';
  }
  if (extension === '.mjs' || extension === '.js') {
    return 'js';
  }
  if (extension === '.css') {
    return 'css';
  }
  if (extension === '.html') {
    return 'html';
  }
  if (extension === '.json') {
    return 'json';
  }
  if (extension === '.yaml') {
    return 'yaml';
  }
  return 'text';
}

```


src/game/EventHorizonGame.ts

```
import {
  Application,
  Container,
  Graphics,
  Sprite,
  Text,
  TextStyle,
  type Texture,
  type Renderer
} from 'pixi.js';
import {
  BLACK_HOLE_X,
  BLACK_HOLE_Y,
  MAX_ENERGY,
  WORLD_HEIGHT,
  WORLD_WIDTH
} from './constants';
import { FixedStepLoop } from './FixedStepLoop';
import { InputHandler, type GestureEndResult, type GesturePoint, type SwipeGesture, type WorldPoint } from './InputHandler';
import { clamp, formatTime } from './math';
import { createSharePoster, type PosterFrame } from './posterizer';
import { submitScore } from './scoreClient';
import { Simulation, type SimulationOptions } from './Simulation';
import type {
  FlybyState,
  OrbState,
  ReplayPayload,
  ShadowArmState,
  SimulationSnapshot,
  SwipeEvent,
  TapEvent
} from './types';

interface OrbView {
  sprite: Sprite;
  glow: Sprite;
}

interface FlybyView {
  sprite: Sprite;
  streak: Graphics;
}

interface TrailFeedback {
  graphics: Graphics;
  label: Text;
  points: WorldPoint[];
  ageMs: number;
  durationMs: number;
  success: boolean;
  canceled: boolean;
}

interface EnergyPulse {
  startX: number;
  startY: number;
  ageMs: number;
  durationMs: number;
}

interface LastGestureDebug {
  kind: 'tap' | 'swipe' | 'canceled';
  screenDistance: number;
  worldDistance: number;
  worldStart?: WorldPoint;
  worldEnd?: WorldPoint;
  result?: SwipeEvent | TapEvent | { message: 'CANCELED'; success: false; target: 'empty' };
  activeTrailCount: number;
}

export class EventHorizonGame {
  private readonly app = new Application<Renderer>();
  private readonly world = new Container();
  private readonly background = new Graphics();
  private readonly tetherLayer = new Graphics();
  private readonly armLayer = new Graphics();
  private readonly blackHole = new Graphics();
  private readonly trailLayer = new Container();
  private readonly liveTrail = new Graphics();
  private readonly energyPulseLayer = new Graphics();
  private readonly feedbackLayer = new Container();
  private readonly collapseLayer = new Graphics();
  private readonly hud = new Container();
  private readonly energyBarBack = new Graphics();
  private readonly energyBarFill = new Graphics();
  private readonly scoreText = new Text({
    text: '0',
    style: new TextStyle({
      fill: '#f7fbff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 52,
      fontWeight: '800'
    })
  });
  private readonly metaText = new Text({
    text: '',
    style: new TextStyle({
      fill: '#9fe7ff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 26,
      fontWeight: '600'
    })
  });
  private readonly energyLabelText = new Text({
    text: 'DARK ENERGY',
    style: new TextStyle({
      fill: '#c7f7ff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 24,
      fontWeight: '800'
    })
  });
  private readonly warningText = new Text({
    text: 'COLLAPSE IMMINENT',
    style: new TextStyle({
      fill: '#ff6a83',

```

```

        fontFamily: 'Inter, system-ui, sans-serif',
        fontSize: 24,
        fontWeight: '800'
    })
});
private readonly tutorialText = new Text({
    text: '',
    style: new TextStyle({
        align: 'center',
        fill: '#ffffff',
        fontFamily: 'Inter, system-ui, sans-serif',
        fontSize: 34,
        fontWeight: '900',
        stroke: { color: '#25103e', width: 5 }
    })
});
private readonly debugText = new Text({
    text: '',
    style: new TextStyle({
        fill: '#d888ff',
        fontFamily: 'SFMono-Regular, Menlo, monospace',
        fontSize: 18,
        fontWeight: '600'
    })
});
private readonly endText = new Text({
    text: '',
    style: new TextStyle({
        align: 'center',
        fill: '#f7fbff',
        fontFamily: 'Inter, system-ui, sans-serif',
        fontSize: 54,
        fontWeight: '800'
    })
});
private readonly sim: Simulation;
private readonly loop: FixedStepLoop;
private input?: InputHandler;
private orbViews: OrbView[] = [];
private flybyViews: FlybyView[] = [];
private frameSamples: PosterFrame[] = [];
private sampleCooldownMs = 0;
private liveGesturePoints: readonly GesturePoint[] = [];
private readonly trails: TrailFeedback[] = [];
private readonly energyPulses: EnergyPulse[] = [];
private readonly orbFlashMs = new Map<number, number>();
private inputDebug = false;
private lastGesture: LastGestureDebug | null = null;
private blackHolePulseMs = 0;
private missPulseMs = 0;
private energyGlowMs = 0;
private onboardingMessage = '';
private onboardingMessageMs = 0;
private scale = 1;
private offsetX = 0;
private offsetY = 0;
private scoreSubmitted = false;
private bestSaved = false;
private paused = false;

constructor(
    private readonly root: HTMLElement,
    options: SimulationOptions
) {
    this.sim = new Simulation(options);
    this.loop = new FixedStepLoop(
        (dtMs) => this.step(dtMs),
        () => this.render()
    );
}

async start(): Promise<void> {
    await this.app.init({
        autoDensity: true,
        autoStart: false,
        backgroundAlpha: 0,
        clearBeforeRender: true,
        hello: false,
        preference: 'webgl',
        preserveDrawingBuffer: true,
        powerPreference: 'high-performance',
        resizeTo: this.root,
        resolution: Math.min(window.devicePixelRatio || 1, 2)
    });

    this.root.appendChild(this.app.canvas);
    this.buildScene();
    this.input = new InputHandler(this.app.canvas, {
        screenToWorld: (clientX, clientY) => this.screenToWorld(clientX, clientY),
        onTap: (point) => this.handleTap(point),
        onSwipe: (gesture) => this.handleSwipe(gesture),
        onGesturePreview: (points) => {
            this.liveGesturePoints = points;
        },
        onGestureEnd: (result) => this.handleGestureEnd(result)
    });
    window.addEventListener('resize', this.resize);
    this.resize();
    this.loop.start();
}

restart(): void {
    this.sim.reset();
    this.scoreSubmitted = false;
    this.bestSaved = false;
    this.frameSamples = [];
    this.sampleCooldownMs = 0;
    this.liveGesturePoints = [];
    this.clearFeedback();
    this.loop.resetClock();
}

forceEnd(): void {
    this.sim.forceEnd();
}

destroy(): void {
    this.loop.stop();
    this.input?.destroy();
}

```

```

window.removeEventListener('resize', this.resize);
this.app.destroy(true, { children: true, texture: true });
}

getReplayPayload(): ReplayPayload {
  return this.sim.getReplayPayload();
}

getSnapshot(): SimulationSnapshot {
  return this.sim.getSnapshot();
}

setPaused(paused: boolean): void {
  this.paused = paused;
  if (!paused) {
    this.loop.resetClock();
  }
}

setInputDebug(enabled: boolean): void {
  this.inputDebug = enabled;
  this.input?.setDebug(enabled);
}

getLastGesture(): LastGestureDebug | null {
  return this.lastGesture
    ? {
        ...this.lastGesture,
        worldStart: this.lastGesture.worldStart ? { ...this.lastGesture.worldStart } : undefined,
        worldEnd: this.lastGesture.worldEnd ? { ...this.lastGesture.worldEnd } : undefined,
        activeTrailCount: this.trails.length
      }
    : null;
}

simulateSwipeWorld(points: readonly WorldPoint[]): SwipeEvent {
  const event = this.sim.applySwipePath(points.map((point, index) => ({ ...point, t: index * 16 })));
  this.handleSwipeFeedback(event, 0, 0, points);
  return event;
}

simulateTapWorld(x: number, y: number): TapEvent {
  const event = this.sim.applyTap(x, y);
  this.lastGesture = {
    kind: 'tap',
    screenDistance: 0,
    worldDistance: 0,
    worldStart: { x, y },
    worldEnd: { x, y },
    result: event,
    activeTrailCount: this.trails.length
  };
  this.addFloatingLabel(this.tapMessage(event), x, y - 42, event.target !== 'empty');
  return event;
}

async exportPoster(): Promise<string> {
  this.sampleFrame('current');
  const snapshot = this.sim.getSnapshot();
  const frames = this.frameSamples.length >= 3 ? this.frameSamples : this.makeFallbackFrames();
  return createSharePoster(frames, {
    score: snapshot.score,
    survivalMs: this.sim.getReplayPayload().survivalMs,
    seed: this.sim.seed,
    phase: snapshot.phase
  });
}

private buildScene(): void {
  this.world.sortableChildren = false;
  this.app.stage.addChild(this.world);
  this.world.addChild(this.background);
  this.world.addChild(this.armLayer);
  this.world.addChild(this.blackHole);
  this.world.addChild(this.tetherLayer);
  this.createOrbViews();
  this.createFlybyViews();
  this.world.addChild(this.trailLayer);
  this.trailLayer.addChild(this.energyPulseLayer, this.liveTrail);
  this.world.addChild(this.feedbackLayer, this.tutorialText);
  this.world.addChild(this.hud);
  this.hud.addChild(
    this.energyBarBack,
    this.energyBarFill,
    this.scoreText,
    this.metaText,
    this.energyLabelText,
    this.warningText,
    this.debugText,
    this.endText
  );
  this.world.addChild(this.collapseLayer);
  this.drawBackground();
  this.drawHudBack();
  this.endText.anchor.set(0.5);
  this.endText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.58);
  this.tutorialText.anchor.set(0.5);
  this.debugText.visible = false;
}

private createOrbViews(): void {
  const orbTexture = this.makeOrbTexture(0x78f2ff, 32, false);
  const glowTexture = this.makeOrbTexture(0xbaf8ff, 76, true);
  for (let index = 0; index < 24; index += 1) {
    const glow = new Sprite(glowTexture);
    glow.anchor.set(0.5);
    glow.blendMode = 'add';
    glow.visible = false;
    const sprite = new Sprite(orbTexture);
    sprite.anchor.set(0.5);
    sprite.visible = false;
    this.world.addChild(glow, sprite);
    this.orbViews.push({ sprite, glow });
  }
}

private createFlybyViews(): void {
  const texture = this.makeOrbTexture(0xffff0a3, 24, false);
  for (let index = 0; index < 6; index += 1) {

```

```

    const streak = new Graphics();
    const sprite = new Sprite(texture);
    sprite.anchor.set(0.5);
    streak.visible = false;
    sprite.visible = false;
    this.world.addChild(streak, sprite);
    this.flybyViews.push({ sprite, streak });
}
}

private makeOrbTexture(color: number, radius: number, soft: boolean): Texture {
    const graphics = new Graphics();
    const alpha = soft ? 0.14 : 0.9;
    graphics.circle(radius, radius).fill({ color, alpha });
    graphics.circle(radius, radius, radius * 0.48).fill({ color: 0xffffffff, alpha: soft ? 0.12 : 0.65 });
    return this.app.renderer.generateTexture(graphics);
}

private drawBackground(): void {
    this.background.clear();
    this.background.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill(0x03040a);

    for (let index = 0; index < 240; index += 1) {
        const x = (index * 197.63) % WORLD_WIDTH;
        const y = (index * 311.19) % WORLD_HEIGHT;
        const depth = (index % 11) / 10;
        const alpha = 0.18 + depth * 0.38;
        const size = 1.1 + depth * 2.2;
        this.background.circle(x, y, size).fill({ color: 0xbfeaff, alpha });
    }

    for (let index = 0; index < 150; index += 1) {
        const t = index / 149;
        const angle = t * Math.PI * 8.8;
        const radius = 82 + t * 690;
        const x = BLACK_HOLE_X + Math.cos(angle) * radius;
        const y = BLACK_HOLE_Y + Math.sin(angle) * radius * 0.57;
        const color = index % 2 === 0 ? 0x305da6 : 0x6e366f;
        this.background.circle(x, y, 3 + t * 7).fill({ color, alpha: 0.07 + (1 - t) * 0.08 });
        this.background.circle(WORLD_WIDTH - x, WORLD_HEIGHT * 0.88 - y * 0.34, 2 + t * 4).fill({
            color: 0x1aaf96,
            alpha: 0.035
        });
    }
}

private drawHudBack(): void {
    this.energyBarBack.clear();
    this.energyBarBack.roundRect(78, WORLD_HEIGHT - 132, WORLD_WIDTH - 156, 38, 7).fill({
        color: 0x061120,
        alpha: 0.86
    });
    this.energyBarBack.roundRect(78, WORLD_HEIGHT - 132, WORLD_WIDTH - 156, 38, 7).stroke({
        color: 0x78f2ff,
        alpha: 0.36,
        width: 2
    });
    this.scoreText.position.set(72, 70);
    this.metaText.position.set(76, 132);
    this.energyLabelText.position.set(86, WORLD_HEIGHT - 166);
    this.warningText.anchor.set(1, 0);
    this.warningText.position.set(WORLD_WIDTH - 86, WORLD_HEIGHT - 166);
    this.debugText.position.set(74, 180);
}

private step(dtMs: number): void {
    this.updateFeedbackTimers(dtMs);
    if (this.paused) {
        return;
    }
    const wasEnded = this.sim.getSnapshot().ended;
    this.sim.step(dtMs);
    const snapshot = this.sim.getSnapshot();
    this.sampleCooldownMs -= dtMs;
    if (this.sampleCooldownMs <= 0 && !snapshot.ended) {
        this.sampleFrame(`${Math.round(snapshot.timeMs)}`);
        this.sampleCooldownMs = 1400;
    }
    if (!wasEnded && snapshot.ended && !this.scoreSubmitted) {
        this.scoreSubmitted = true;
        this.saveBestRun();
        void submitScore(this.sim.getReplayPayload());
    }
}

private render(): void {
    const snapshot = this.sim.getSnapshot();
    this.world.position.set(this.offsetX, this.offsetY);
    this.world.scale.set(this.scale);
    this.renderShadowArms(snapshot.shadowArms, snapshot.phase);
    this.renderBlackHole(snapshot);
    this.renderTethers(snapshot.orbs);
    this.renderOrbs(snapshot.orbs, snapshot.collapseT);
    this.renderFlybys(snapshot.flybys);
    this.renderTrails();
    this.renderEnergyPulses();
    this.renderTutorial(snapshot);
    this.renderHud(snapshot);
    this.renderDebug(snapshot);
    this.renderCollapse(snapshot);
    this.app.render();
}

private renderShadowArms(arms: readonly ShadowArmState[], phase: number): void {
    this.armLayer.clear();
    if (phase < 2) {
        return;
    }
    for (const arm of arms) {
        const length = 420 + phase * 90;
        const endX = BLACK_HOLE_X + Math.cos(arm.angle) * length;
        const endY = BLACK_HOLE_Y + Math.sin(arm.angle) * length;
        const alpha = arm.stunMs > 0 ? 0.12 : 0.12 + arm.intensity * 0.32;
        this.armLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
        this.armLayer.lineTo(endX, endY);
        this.armLayer.stroke({ color: arm.stunMs > 0 ? 0x8ff9ff : 0x1b102b, alpha, width: 50 + phase * 7 });
        this.armLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
        this.armLayer.lineTo(endX, endY);
        this.armLayer.stroke({ color: 0x8d62ff, alpha: alpha * 0.45, width: 8 });
    }
}

```

```

    }
}

private renderBlackHole(snapshot: SimulationSnapshot): void {
    this.blackHole.clear();
    const phaseT = (snapshot.phase - 1) / 3;
    const pulse = Math.sin(snapshot.timeMs * 0.004) * 0.5 + 0.5;
    const harvestPulse = clamp(this.blackHolePulseMs / 320, 0, 1);
    const missPulse = clamp(this.missPulseMs / 260, 0, 1);
    const radius = 80 + phaseT * 54 + snapshot.collapseT * 260 - harvestPulse * 12 + missPulse * 10;
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 2.2).fill({
        color: 0x0a1424,
        alpha: 0.08 + phaseT * 0.13
    });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 1.25).stroke({
        color: harvestPulse > 0 ? 0xd9fbff : missPulse > 0 ? 0xff6a83 : 0x6bcfff,
        alpha: 0.14 + phaseT * 0.26 + harvestPulse * 0.3 + missPulse * 0.18,
        width: 7 + pulse * 6 + harvestPulse * 10
    });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius).fill({ color: 0x000000, alpha: 0.82 });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 0.38).fill({ color: 0x03040a, alpha: 1 });
}

private renderTethers(orbs: readonly OrbState[]): void {
    this.tetherLayer.clear();
    for (const orb of orbs) {
        if (!orb.active) {
            continue;
        }
        const flash = clamp((this.orbFlashMs.get(orb.id) ?? 0) / 650, 0, 1);
        const tutorialBoost = orb.tutorial ? 0.42 : 0;
        const alpha = clamp(orb.captured ? 0.78 : orb.frozenMs > 0 ? 0.56 : 0.24 + tutorialBoost + flash * 0.42, 0, 0.95);
        this.tetherLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
        this.tetherLayer.lineTo(orb.x, orb.y);
        this.tetherLayer.stroke({
            color: orb.captured ? 0xd9fbff : 0x5bc7ff,
            alpha,
            width: orb.captured ? 8 : orb.tutorial ? 7 : orb.frozenMs > 0 ? 5 : 3
        });
        if (orb.tutorial || flash > 0) {
            const outerX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * 0.46;
            const outerY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * 0.46;
            this.tetherLayer.moveTo(outerX, outerY);
            this.tetherLayer.lineTo(orb.x, orb.y);
            this.tetherLayer.stroke({
                color: flash > 0 ? 0xffffffff : 0xd267ff,
                alpha: 0.42 + flash * 0.38,
                width: 18 + flash * 10
            });
        }
    }
}

private renderOrbs(orbs: readonly OrbState[], collapseT: number): void {
    for (let index = 0; index < this.orbViews.length; index += 1) {
        const orb = orbs[index];
        const view = this.orbViews[index];
        if (!orb?.active) {
            view.sprite.visible = false;
            view.glow.visible = false;
            continue;
        }
        const bendX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * (1 - collapseT * 0.88);
        const bendY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * (1 - collapseT * 0.88);
        const frozen = orb.frozenMs > 0;
        view.sprite.visible = true;
        view.glow.visible = true;
        view.sprite.position.set(bendX, bendY);
        view.glow.position.set(bendX, bendY);
        const scale = orb.radius / 32;
        const flash = clamp((this.orbFlashMs.get(orb.id) ?? 0) / 650, 0, 1);
        const tutorial = orb.tutorial ? 1 : 0;
        view.sprite.scale.set(scale * (orb.captured ? 1.42 : frozen ? 1.24 : 1 + tutorial * 0.12 + flash * 0.18));
        view.glow.scale.set(scale * (orb.captured ? 2.1 : frozen ? 1.62 : 1.28 + tutorial * 0.42 + flash * 0.46));
        view.glow.alpha = clamp(orb.captured ? 1 : frozen ? 0.96 : 0.48 + tutorial * 0.32 + flash * 0.3, 0, 1);
        view.sprite.alpha = 1 - collapseT * 0.45;
    }
}

private renderFlybys(flybys: readonly FlybyState[]): void {
    for (let index = 0; index < this.flybyViews.length; index += 1) {
        const flyby = flybys[index];
        const view = this.flybyViews[index];
        if (!flyby?.active) {
            view.sprite.visible = false;
            view.streak.visible = false;
            continue;
        }
        view.sprite.visible = true;
        view.streak.visible = true;
        view.sprite.position.set(flyby.x, flyby.y);
        view.sprite.scale.set(flyby.radius / 24);
        view.streak.clear();
        view.streak.moveTo(flyby.x - flyby.vx * 0.08, flyby.y - flyby.vy * 0.08);
        view.streak.lineTo(flyby.x, flyby.y);
        view.streak.stroke({ color: 0xffff0a3, alpha: 0.55, width: 8 });
    }
}

private renderHud(snapshot: SimulationSnapshot): void {
    const width = (WORLD_WIDTH - 172) * clamp(snapshot.energy / MAX_ENERGY, 0, 1);
    const barColor = snapshot.phase >= 4 ? 0xff5d73 : snapshot.phase >= 3 ? 0xffc857 : 0x67f4ff;
    const glow = clamp(this.energyGlowMs / 700, 0, 1);
    this.energyBarFill.clear();
    if (glow > 0) {
        this.energyBarFill.roundRect(70, WORLD_HEIGHT - 140, WORLD_WIDTH - 140, 54, 12).stroke({
            color: 0xd267ff,
            alpha: glow * 0.75,
            width: 8 + glow * 8
        });
    }
    this.energyBarFill.roundRect(86, WORLD_HEIGHT - 124, width, 22, 6).fill({ color: barColor, alpha: 0.95 });
    this.energyBarFill.roundRect(86, WORLD_HEIGHT - 124, width, 22, 6).fill({ color: 0xffffffff, alpha: 0.14 });
    this.scoreText.text = snapshot.score.toString();
    this.metaText.text = `${formatTime(this.sim.getReplayPayload().survivalMs)} PHASE ${snapshot.phase} x${snapshot.streak}`;
    this.warningText.visible = snapshot.energy < 25 && !snapshot.ended;
    this.endText.visible = snapshot.ended;
    this.endText.text = snapshot.ended
    ? `GALAXY COLLAPSED\n${snapshot.score} • ${formatTime(this.sim.getReplayPayload().survivalMs)}`

```

```

    : '';
}

private renderCollapse(snapshot: SimulationSnapshot): void {
    this.collapseLayer.clear();
    if (!snapshot.ended) {
        return;
    }
    const t = snapshot.collapseT;
    this.world.scale.set(this.scale * (1 - t * 0.06));
    this.world.position.set(this.offsetX + WORLD_WIDTH * this.scale * t * 0.03, this.offsetY + WORLD_HEIGHT * this.scale * t * 0.025);
    this.collapseLayer.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill({ color: 0x000000, alpha: clamp((t - 0.42) / 0.58, 0, 1) });
}

private handleTap(point: GesturePoint): void {
    const event = this.sim.applyTap(point.x, point.y);
    this.lastGesture = {
        kind: 'tap',
        screenDistance: 0,
        worldDistance: 0,
        worldStart: { x: point.x, y: point.y },
        worldEnd: { x: point.x, y: point.y },
        result: event,
        activeTrailCount: this.trails.length
    };
    this.addFloatingLabel(this.tapMessage(event), point.x, point.y - 44, event.target !== 'empty');
}

private handleSwipe(gesture: SwipeGesture): void {
    const event = this.sim.applySwipeGesture(gesture);
    this.handleSwipeFeedback(event, gesture.screenDistance, gesture.worldDistance, gesture.points);
}

private handleGestureEnd(result: GestureEndResult): void {
    this.liveGesturePoints = [];
    if (result.kind !== 'canceled' || !result.gesture) {
        return;
    }
    this.lastGesture = {
        kind: 'canceled',
        screenDistance: result.gesture.screenDistance,
        worldDistance: result.gesture.worldDistance,
        worldStart: { x: result.gesture.start.x, y: result.gesture.start.y },
        worldEnd: { x: result.gesture.end.x, y: result.gesture.end.y },
        result: { message: 'CANCELED', success: false, target: 'empty' },
        activeTrailCount: this.trails.length
    };
    this.addSwipeTrail(result.gesture.points, 'CANCELED', false, true);
}

private handleSwipeFeedback(
    event: SwipeEvent,
    screenDistance: number,
    worldDistance: number,
    sourcePoints: readonly WorldPoint[]
): void {
    const message = event.success
        ? `${event.message} ${event.scoreDelta ?? 0}`
        : event.message;
    this.addSwipeTrail(event.path.length > 0 ? event.path : sourcePoints, message, event.success, false);
    const start = event.path[0] ?? sourcePoints[0] ?? { x: event.x1, y: event.y1 };
    const end = event.path[event.path.length - 1] ?? sourcePoints[sourcePoints.length - 1] ?? { x: event.x2, y: event.y2 };
    this.lastGesture = {
        kind: 'swipe',
        screenDistance,
        worldDistance,
        worldStart: start,
        worldEnd: end,
        result: event,
        activeTrailCount: this.trails.length
    };

    if (event.success) {
        if (event.orbId !== undefined) {
            this.orbFlashMs.set(event.orbId, 650);
            const orb = this.sim.getSnapshot().orbs.find((candidate) => candidate.id === event.orbId);
            this.energyPulses.push({
                startX: orb?.x ?? end.x,
                startY: orb?.y ?? end.y,
                ageMs: 0,
                durationMs: 620
            });
        }
        this.energyGlowMs = 700;
        this.blackHolePulseMs = 320;
        this.onboardingMessage = this.sim.getSnapshot().harvestCount === 1 ? 'DARK ENERGY CAPTURED' : '';
        this.onboardingMessageMs = this.onboardingMessage ? 1800 : this.onboardingMessageMs;
        if ('vibrate' in navigator) {
            navigator.vibrate?.(12);
        }
    } else {
        this.missPulseMs = 260;
    }
}

private addSwipeTrail(
    points: readonly WorldPoint[],
    labelText: string,
    success: boolean,
    canceled: boolean
): void {
    if (points.length === 0) {
        return;
    }
    const graphics = new Graphics();
    const label = new Text({
        text: labelText,
        style: new TextStyle({
            align: 'center',
            fill: success ? 'ffffff' : canceled ? '#9fb5c4' : '#ff8ba1',
            fontFamily: 'Inter, system-ui, sans-serif',
            fontSize: 30,
            fontWeight: '900',
            stroke: { color: '#15051f', width: 5 }
        })
    });
    label.anchor.set(0.5);
    const midpoint = points[Math.floor(points.length / 2)];
    label.position.set(midpoint.x, midpoint.y - 54);
}

```

```

this.trailLayer.addChild(graphics);
this.trailLayer.addChild(this.liveTrail);
this.feedbackLayer.addChild(label);
this.trails.push({
  graphics,
  label,
  points: points.map((point) => ({ x: point.x, y: point.y })),
  ageMs: 0,
  durationMs: success ? 820 : 680,
  success,
  canceled
});

while (this.trails.length > 8) {
  this.removeTrail(this.trails[0]!);
}

private addFloatingLabel(text: string, x: number, y: number, success: boolean): void {
  const points = [
    { x: x - 4, y },
    { x: x + 4, y }
  ];
  this.addSwipeTrail(points, text, success, false);
}

private tapMessage(event: TapEvent): string {
  if (event.target === 'orb') {
    return 'STABILIZED +8';
  }
  if (event.target === 'flyby') {
    return 'BONUS';
  }
  if (event.target === 'arm') {
    return 'STUNNED';
  }
  return 'MISS';
}

private renderTrails(): void {
  this.liveTrail.clear();
  for (const trail of this.trails) {
    const alpha = clamp(1 - trail.ageMs / trail.durationMs, 0, 1);
    trail.graphics.clear();
    this.drawTrailPath(trail.graphics, trail.points, alpha, trail.success, trail.canceled);
    trail.label.alpha = alpha;
    trail.label.y -= 0.28;
  }

  if (this.liveGesturePoints.length > 0) {
    this.drawTrailPath(this.liveTrail, this.liveGesturePoints, 0.9, true, false);
  }
}

private drawTrailPath(
  graphics: Graphics,
  points: readonly WorldPoint[],
  alpha: number,
  success: boolean,
  canceled: boolean
): void {
  if (points.length === 0) {
    return;
  }
  const color = canceled ? 0x7d90a2 : success ? 0xd267ff : 0xff6a83;
  const core = success ? 0xffffffff : color;
  graphics.moveTo(points[0].x, points[0].y);
  for (let index = 1; index < points.length; index += 1) {
    graphics.lineTo(points[index].x, points[index].y);
  }
  graphics.stroke({ color, alpha: alpha * 0.42, width: 22 });
  graphics.moveTo(points[0].x, points[0].y);
  for (let index = 1; index < points.length; index += 1) {
    graphics.lineTo(points[index].x, points[index].y);
  }
  graphics.stroke({ color: core, alpha: alpha * 0.95, width: 6 });
  const start = points[0];
  const end = points[points.length - 1];
  graphics.circle(start.x, start.y, 11).fill({ color: core, alpha: alpha * 0.9 });
  graphics.circle(end.x, end.y, success ? 18 : 13).stroke({ color: core, alpha: alpha, width: success ? 5 : 3 });
}

private renderEnergyPulses(): void {
  this.energyPulseLayer.clear();
  for (const pulse of this.energyPulses) {
    const t = clamp(pulse.ageMs / pulse.durationMs, 0, 1);
    const targetX = WORLD_WIDTH / 2;
    const targetY = WORLD_HEIGHT - 112;
    const x = pulse.startX + (targetX - pulse.startX) * t;
    const y = pulse.startY + (targetY - pulse.startY) * t;
    const alpha = 1 - t;
    this.energyPulseLayer.moveTo(pulse.startX, pulse.startY);
    this.energyPulseLayer.lineTo(x, y);
    this.energyPulseLayer.stroke({ color: 0x67f4ff, alpha: alpha * 0.34, width: 5 });
    this.energyPulseLayer.circle(x, y, 10 + (1 - t) * 10).fill({ color: 0xd267ff, alpha: alpha * 0.78 });
  }
}

private renderTutorial(snapshot: SimulationSnapshot): void {
  const tutorialOrb = snapshot.orbs.find((orb) => orb.active && orb.tutorial && !orb.captured);
  if (tutorialOrb && snapshot.harvestCount === 0) {
    this.tutorialText.visible = true;
    this.tutorialText.text = 'SWIPE THE TETHER';
    this.tutorialText.position.set(tutorialOrb.x, tutorialOrb.y - tutorialOrb.radius - 76);
    return;
  }

  if (this.onboardingMessageMs > 0 && this.onboardingMessage) {
    this.tutorialText.visible = true;
    this.tutorialText.text = this.onboardingMessage;
    this.tutorialText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.31);
    return;
  }

  if (snapshot.harvestCount > 0 && snapshot.timeMs < 22000) {
    this.tutorialText.visible = true;
    this.tutorialText.text = 'TAP TO STABILIZE';
    this.tutorialText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.31);
    return;
  }
}

```

```

    }
    this.tutorialText.visible = false;
  }

  private renderDebug(snapshot: SimulationSnapshot): void {
    this.debugText.visible = this.inputDebug;
    if (!this.inputDebug) {
      return;
    }
    const input = this.input?.getDebugInfo();
    const last = this.lastGesture;
    const start = input?.worldStart;
    const end = input?.worldEnd;
    this.debugText.text = [
      `input: ${input?.lastPointerType ?? 'none'} -> ${input?.lastGestureKind ?? 'none'}`,
      `screen: ${Math.round(input?.lastScreenDistance ?? 0)}px world: ${Math.round(input?.lastWorldDistance ?? 0)}`,
      `start: ${start ? `${Math.round(start.x)}, ${Math.round(start.y)}` : '—'}`,
      `end: ${end ? `${Math.round(end.x)}, ${Math.round(end.y)}` : '—'}`,
      `result: ${last?.result ? `${'target' in last.result ? last.result.target : 'empty'} / ${'message' in last.result ? last.result.message : 'TAP'}` : 'none'}`,
      `energy: ${Math.round(snapshot.energy)} score: ${snapshot.score}`
    ].join('\n');
  }

  private updateFeedbackTimers(dtMs: number): void {
    this.blackHolePulseMs = Math.max(0, this.blackHolePulseMs - dtMs);
    this.missPulseMs = Math.max(0, this.missPulseMs - dtMs);
    this.energyGlowMs = Math.max(0, this.energyGlowMs - dtMs);
    this.onboardingMessageMs = Math.max(0, this.onboardingMessageMs - dtMs);
    for (const [orbId, ms] of this.orbFlashMs) {
      const next = ms - dtMs;
      if (next <= 0) {
        this.orbFlashMs.delete(orbId);
      } else {
        this.orbFlashMs.set(orbId, next);
      }
    }
    for (let index = this.trails.length - 1; index >= 0; index -= 1) {
      const trail = this.trails[index];
      trail.ageMs += dtMs;
      if (trail.ageMs >= trail.durationMs) {
        this.removeTrail(trail);
      }
    }
    for (let index = this.energyPulses.length - 1; index >= 0; index -= 1) {
      const pulse = this.energyPulses[index];
      pulse.ageMs += dtMs;
      if (pulse.ageMs >= pulse.durationMs) {
        this.energyPulses.splice(index, 1);
      }
    }
  }

  private removeTrail(trail: TrailFeedback): void {
    const index = this.trails.indexOf(trail);
    if (index >= 0) {
      this.trails.splice(index, 1);
    }
    trail.graphics.destroy();
    trail.label.destroy();
  }

  private clearFeedback(): void {
    for (const trail of [...this.trails]) {
      this.removeTrail(trail);
    }
    this.energyPulses.length = 0;
    this.energyPulseLayer.clear();
    this.liveTrail.clear();
    this.orbFlashMs.clear();
    this.lastGesture = null;
    this.onboardingMessage = '';
    this.onboardingMessageMs = 0;
  }

  private saveBestRun(): void {
    if (this.bestSaved) {
      return;
    }
    this.bestSaved = true;
    try {
      const replay = this.sim.getReplayPayload();
      const bestSurvival = Number(localStorage.getItem('eventHorizon.bestSurvivalMs') ?? 0);
      const bestScore = Number(localStorage.getItem('eventHorizon.bestScore') ?? 0);
      if (replay.survivalMs > bestSurvival) {
        localStorage.setItem('eventHorizon.bestSurvivalMs', String(replay.survivalMs));
      }
      if (replay.score > bestScore) {
        localStorage.setItem('eventHorizon.bestScore', String(replay.score));
      }
    } catch {
      // Private browsing or locked-down WebViews can reject localStorage.
    }
  }

  private readonly resize = (): void => {
    const rect = this.root.getBoundingClientRect();
    this.scale = Math.min(rect.width / WORLD_WIDTH, rect.height / WORLD_HEIGHT);
    const viewWidth = WORLD_WIDTH * this.scale;
    const viewHeight = WORLD_HEIGHT * this.scale;
    this.offsetX = (rect.width - viewWidth) / 2;
    this.offsetY = (rect.height - viewHeight) / 2;
    this.world.position.set(this.offsetX, this.offsetY);
    this.world.scale.set(this.scale);
  };

  private screenToWorld(clientX: number, clientY: number): WorldPoint {
    const rect = this.app.canvas.getBoundingClientRect();
    return {
      x: clamp(clientX - rect.left - this.offsetX) / this.scale, 0, WORLD_WIDTH),
      y: clamp(clientY - rect.top - this.offsetY) / this.scale, 0, WORLD_HEIGHT)
    };
  }

  private sampleFrame(label: string): void {
    try {
      const dataUrl = this.app.canvas.toDataURL('image/png', 0.72);
      this.frameSamples.push({ dataUrl, label });
      if (this.frameSamples.length > 3) {

```



```

        this.frameSamples.shift();
    }
} catch {
    this.frameSamples = this.makeFallbackFrames();
}
}

private makeFallbackFrames(): PosterFrame[] {
    const snapshot = this.sim.getSnapshot();
    return [0, 1, 2].map((index) => ({
        dataUrl: this.makeMockFrame(snapshot, index),
        label: `mock-${index}`
    }));
}

private makeMockFrame(snapshot: SimulationSnapshot, index: number): string {
    const canvas = document.createElement('canvas');
    canvas.width = 540;
    canvas.height = 960;
    const context = canvas.getContext('2d');
    if (!context) {
        return '';
    }
    context.fillStyle = '#03040a';
    context.fillRect(0, 0, canvas.width, canvas.height);
    context.fillStyle = index === 0 ? '#143c5c' : index === 1 ? '#3c285f' : '#501421';
    context.beginPath();
    context.arc(270, 410, 180 + snapshot.phase * 24, 0, Math.PI * 2);
    context.fill();
    context.fillStyle = '#000000';
    context.beginPath();
    context.arc(270, 410, 62 + snapshot.phase * 15, 0, Math.PI * 2);
    context.fill();
    context.fillStyle = '#f7fbff';
    context.font = '700 54px system-ui';
    context.fillText(String(snapshot.score), 52, 830);
    return canvas.toDataURL('image/png', 0.8);
}
}

```

src/game/InputHandler.ts

```

import {
    makeSwipeGesture,
    screenPathLength,
    worldPathLength,
    type GesturePoint,
    type SwipeGesture,
    type WorldPoint
} from './gestures';

export type { GesturePoint, SwipeGesture, WorldPoint } from './gestures';

export interface GestureEndResult {
    kind: 'tap' | 'swipe' | 'canceled';
    gesture?: SwipeGesture;
    point?: GesturePoint;
    canceled: boolean;
}

export interface InputDebugInfo {
    enabled: boolean;
    lastPointerEventType: string;
    lastGestureKind: GestureEndResult['kind'] | 'none';
    lastScreenDistance: number;
    lastWorldDistance: number;
    worldStart?: WorldPoint;
    worldEnd?: WorldPoint;
}

export interface InputCallbacks {
    screenToWorld: (clientX: number, clientY: number) => WorldPoint;
    onTap: (point: GesturePoint) => void;
    onSwipe: (gesture: SwipeGesture) => void;
    onGesturePreview?: (points: readonly GesturePoint[]) => void;
    onGestureEnd?: (result: GestureEndResult) => void;
}

export class InputHandler {
    private activePointerId: number | null = null;
    private activeTouchId: number | null = null;
    private points: GesturePoint[] = [];
    private debugInfo: InputDebugInfo = {
        enabled: false,
        lastPointerEventType: 'none',
        lastGestureKind: 'none',
        lastScreenDistance: 0,
        lastWorldDistance: 0
    };
    private readonly supportsPointerEvents = typeof window !== 'undefined' && 'PointerEvent' in window;
    private readonly swipeThresholdPx = 20;

    constructor(
        private readonly target: HTMLElement,
        private readonly callbacks: InputCallbacks
    ) {
        this.target.addEventListener('contextmenu', this.preventDefault, { passive: false });
        this.target.addEventListener('selectstart', this.preventDefault, { passive: false });

        if (this.supportsPointerEvents) {
            this.target.addEventListener('pointerdown', this.handlePointerDown, { passive: false });
            this.target.addEventListener('pointermove', this.handlePointerMove, { passive: false });
            this.target.addEventListener('pointerup', this.handlePointerUp, { passive: false });
            this.target.addEventListener('pointercancel', this.handlePointerCancel, { passive: false });
        } else {
            this.target.addEventListener('touchstart', this.handleTouchStart, { passive: false });
            this.target.addEventListener('touchmove', this.handleTouchMove, { passive: false });
            this.target.addEventListener('touchend', this.handleTouchEnd, { passive: false });
            this.target.addEventListener('touchcancel', this.handleTouchCancel, { passive: false });
        }
    }

    destroy(): void {
        this.target.removeEventListener('contextmenu', this.preventDefault);
        this.target.removeEventListener('selectstart', this.preventDefault);
        this.target.removeEventListener('pointerdown', this.handlePointerDown);
    }
}

```

```

this.target.removeEventListener('pointermove', this.handlePointerMove);
this.target.removeEventListener('pointerup', this.handlePointerUp);
this.target.removeEventListener('pointercancel', this.handlePointerCancel);
this.target.removeEventListener('touchstart', this.handleTouchStart);
this.target.removeEventListener('touchmove', this.handleTouchMove);
this.target.removeEventListener('touchend', this.handleTouchEnd);
this.target.removeEventListener('touchcancel', this.handleTouchCancel);
}

setDebug(enabled: boolean): void {
  this.debugInfo.enabled = enabled;
}

getDebugInfo(): InputDebugInfo {
  return {
    ...this.debugInfo,
    worldStart: this.debugInfo.worldStart ? { ...this.debugInfo.worldStart } : undefined,
    worldEnd: this.debugInfo.worldEnd ? { ...this.debugInfo.worldEnd } : undefined
  };
}

private readonly preventDefault = (event: Event): void => {
  event.preventDefault();
};

private readonly handlePointerDown = (event: PointerEvent): void => {
  this.recordEventType(event.type);
  event.preventDefault();
  if (this.activePointerId !== null || (event.pointerType === 'mouse' && event.button !== 0)) {
    return;
  }
  this.activePointerId = event.pointerId;
  try {
    this.target.setPointerCapture?.(event.pointerId);
  } catch {
    // Some mobile browsers throw when capture races element removal or cancellation.
  }
  this.startGesture(this.makePoint(event.clientX, event.clientY, event.timeStamp));
};

private readonly handlePointerMove = (event: PointerEvent): void => {
  if (event.pointerId !== this.activePointerId) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  const coalesced = this.getCoalescedPointerEvents(event);
  for (const sample of coalesced) {
    this.addPoint(this.makePoint(sample.clientX, sample.clientY, sample.timeStamp));
  }
  this.previewGesture();
};

private readonly handlePointerUp = (event: PointerEvent): void => {
  if (event.pointerId !== this.activePointerId) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  this.addPoint(this.makePoint(event.clientX, event.clientY, event.timeStamp));
  this.finishGesture(false);
  this.resetPointer(event.pointerId);
};

private readonly handlePointerCancel = (event: PointerEvent): void => {
  if (event.pointerId !== this.activePointerId) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  this.addPoint(this.makePoint(event.clientX, event.clientY, event.timeStamp));
  this.finishGesture(true);
  this.resetPointer(event.pointerId);
};

private readonly handleTouchStart = (event: TouchEvent): void => {
  this.recordEventType(event.type);
  event.preventDefault();
  if (this.activeTouchId !== null || event.changedTouches.length === 0) {
    return;
  }
  const touch = event.changedTouches[0];
  this.activeTouchId = touch.identifier;
  this.startGesture(this.makePoint(touch.clientX, touch.clientY, event.timeStamp));
};

private readonly handleTouchMove = (event: TouchEvent): void => {
  const touch = this.findActiveTouch(event.changedTouches);
  if (!touch) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  this.addPoint(this.makePoint(touch.clientX, touch.clientY, event.timeStamp));
  this.previewGesture();
};

private readonly handleTouchEnd = (event: TouchEvent): void => {
  const touch = this.findActiveTouch(event.changedTouches);
  if (!touch) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  this.addPoint(this.makePoint(touch.clientX, touch.clientY, event.timeStamp));
  this.finishGesture(false);
  this.activeTouchId = null;
};

private readonly handleTouchCancel = (event: TouchEvent): void => {
  const touch = this.findActiveTouch(event.changedTouches);
  if (!touch) {
    return;
  }
  this.recordEventType(event.type);
  event.preventDefault();
  this.addPoint(this.makePoint(touch.clientX, touch.clientY, event.timeStamp));
  this.finishGesture(true);
  this.activeTouchId = null;
};

```

```

};

private startGesture(point: GesturePoint): void {
  this.points = [point];
  this.debugInfo.lastGestureKind = 'none';
  this.updateDebugDistances();
  this.previewGesture();
}

private addPoint(point: GesturePoint): void {
  const previous = this.points[this.points.length - 1];
  if (previous && previous.screenX === point.screenX && previous.screenY === point.screenY) {
    return;
  }
  this.points.push(point);
  this.updateDebugDistances();
}

private previewGesture(): void {
  this.callbacks.onGesturePreview?.(this.points);
}

private finishGesture(canceled: boolean): void {
  if (this.points.length === 0) {
    return;
  }

  const gesture = this.points.length > 1 ? makeSwipeGesture(this.points) : undefined;
  if (canceled) {
    this.debugInfo.lastGestureKind = 'canceled';
    this.callbacks.onGestureEnd?.({ kind: 'canceled', gesture, point: this.points[0], canceled: true });
    this.points = [];
    return;
  }

  if (gesture && gesture.screenDistance >= this.swipeThresholdPx) {
    this.debugInfo.lastGestureKind = 'swipe';
    this.callbacks.onSwipe(gesture);
    this.callbacks.onGestureEnd?.({ kind: 'swipe', gesture, canceled: false });
  } else {
    const point = this.points[this.points.length - 1];
    this.debugInfo.lastGestureKind = 'tap';
    this.callbacks.onTap(point);
    this.callbacks.onGestureEnd?.({ kind: 'tap', gesture, point, canceled: false });
  }
  this.points = [];
}

private resetPointer(pointerId: number): void {
  try {
    this.target.releasePointerCapture?.(pointerId);
  } catch {
    // Ignore capture-release races; the gesture has already been finalized.
  }
  this.activePointerId = null;
}

private findActiveTouch(touches: TouchList): Touch | undefined {
  for (let index = 0; index < touches.length; index += 1) {
    const touch = touches.item(index);
    if (touch && touch.identifier === this.activeTouchId) {
      return touch;
    }
  }
  return undefined;
}

private getCoalescedPointerEvents(event: PointerEvent): readonly PointerEvent[] {
  if (typeof event.getCoalescedEvents !== 'function') {
    return [event];
  }
  const coalesced = event.getCoalescedEvents();
  return coalesced.length > 0 ? coalesced : [event];
}

private makePoint(clientX: number, clientY: number, timeStamp: number): GesturePoint {
  const world = this.callbacks.screenToWorld(clientX, clientY);
  return {
    x: world.x,
    y: world.y,
    screenX: clientX,
    screenY: clientY,
    t: timeStamp || performance.now()
  };
}

private updateDebugDistances(): void {
  const first = this.points[0];
  const last = this.points[this.points.length - 1];
  this.debugInfo.lastScreenDistance = screenPathLength(this.points);
  this.debugInfo.lastWorldDistance = worldPathLength(this.points);
  this.debugInfo.worldStart = first ? { x: first.x, y: first.y } : undefined;
  this.debugInfo.worldEnd = last ? { x: last.x, y: last.y } : undefined;
}

private recordEventType(type: string): void {
  this.debugInfo.lastPointerEventType = type;
}
}

```

src/game/Simulation.ts

```

import {
  BLACK_HOLE_X,
  BLACK_HOLE_Y,
  FLYBY_POOL_SIZE,
  INITIAL_ENERGY,
  MAX_ENERGY,
  ORB_POOL_SIZE,
  WORLD_HEIGHT,
  WORLD_WIDTH
} from './constants';
import { clamp, distanceSquared, distanceToSegmentSquared } from './math';
import {
  quantizeGesturePath,
  segmentToSegmentDistanceSquared,

```

```

    type GesturePathPoint
  } from './gestures';
import { createSeededRandom, type RandomSource } from './rng';
import type {
  FlybyState,
  GameState,
  OrbState,
  PhaseTransition,
  ReplayPayload,
  ShadowArmState,
  SimulationSnapshot,
  SwipeEvent,
  TapEvent
} from './types';

const ORB_CAPTURE_RADIUS = 82;
const FLYBY_CAPTURE_RADIUS = 58;
const EVENT_HORIZON_RADIUS = 98;
const ARM_LENGTH = 760;
const ARM_HIT_WIDTH = 46;
const TETHER_HIT_WIDTH = 68;
const TETHER_OUTER_START = 0.46;

export interface SimulationOptions {
  seed: string;
  startedAt: number;
}

export interface TimedInput {
  kind: 'tap' | 'swipe';
  t: number;
  x: number;
  y: number;
  x2?: number;
  y2?: number;
  path?: GesturePathPoint[];
}

interface TetherHit {
  orb: OrbState;
  distanceSq: number;
}

export class Simulation {
  readonly seed: string;
  readonly startedAt: number;
  private rng: RandomSource;
  private nextOrbId = 1;
  private nextFlybyId = 1;
  private orbSpawnMs = 900;
  private flybySpawnMs = 2600;
  private readonly tapEvents: TapEvent[] = [];
  private readonly swipeEvents: SwipeEvent[] = [];
  private readonly phaseTransitions: PhaseTransition[] = [];
  private readonly orbs: OrbState[] = [];
  private readonly flybys: FlybyState[] = [];
  private readonly shadowArms: ShadowArmState[] = [];
  private timeMs = 0;
  private score = 0;
  private energy = INITIAL_ENERGY;
  private energyCaptured = 0;
  private streak = 0;
  private maxStreak = 0;
  private harvestCount = 0;
  private phase: GameState = 1;
  private ended = false;
  private collapseMs = 0;

  constructor(options: SimulationOptions) {
    this.seed = options.seed;
    this.startedAt = options.startedAt;
    this.rng = createSeededRandom(options.seed);
    this.initPools();
    this.phaseTransitions.push({ t: 0, phase: 1, energy: this.energy });
  }

  reset(): void {
    this.rng = createSeededRandom(this.seed);
    this.nextOrbId = 1;
    this.nextFlybyId = 1;
    this.orbSpawnMs = 900;
    this.flybySpawnMs = 2600;
    this.tapEvents.length = 0;
    this.swipeEvents.length = 0;
    this.phaseTransitions.length = 0;
    this.timeMs = 0;
    this.score = 0;
    this.energy = INITIAL_ENERGY;
    this.energyCaptured = 0;
    this.streak = 0;
    this.maxStreak = 0;
    this.harvestCount = 0;
    this.phase = 1;
    this.ended = false;
    this.collapseMs = 0;
    this.initPools();
    this.phaseTransitions.push({ t: 0, phase: 1, energy: this.energy });
  }

  step(dtMs: number): void {
    const dt = dtMs / 1000;
    if (this.ended) {
      this.timeMs += dtMs;
      this.collapseMs = Math.min(this.collapseMs + dtMs, 2200);
      return;
    }

    this.timeMs += dtMs;
    const onboardingGrace = this.timeMs < 30000 || this.harvestCount === 0;
    const drain = onboardingGrace ? 0.14 + this.phase * 0.025 : 0.28 + this.phase * 0.05;
    this.energy = clamp(this.energy - dt * drain, 0, MAX_ENERGY);
    this.spawnOrbs(dtMs);
    this.spawnFlybys(dtMs);
    this.updateShadowArms(dtMs);
    this.updateOrbs(dtMs, dt);
    this.updateFlybys(dtMs, dt);
    this.updatePhase();

    if (this.energy <= 0) {

```

```

    this.endRun();
  }
}

applyTap(x: number, y: number): TapEvent {
  let target: TapEvent['target'] = 'empty';
  const flyby = this.findFlyby(x, y);
  if (flyby) {
    this.collectFlyby(flyby);
    target = 'flyby';
  } else {
    const orb = this.findOrb(x, y, ORB_CAPTURE_RADIUS);
    if (orb) {
      orb.frozenMs = Math.max(orb.frozenMs, 520);
      this.score += 8 + this.phase;
      target = 'orb';
    } else if (this.stunArm(x, y)) {
      target = 'arm';
    }
  }

  const event: TapEvent = { t: Math.round(this.timeMs), x: Math.round(x), y: Math.round(y), target };
  this.tapEvents.push(event);
  return event;
}

applySwipeGesture(gesture: { points: readonly GesturePathPoint[] }): SwipeEvent {
  return this.applySwipePath(gesture.points);
}

applySwipePath(points: readonly GesturePathPoint[]): SwipeEvent {
  const path = quantizeGesturePath(points, 24);
  const start = path[0] ?? {
    x: Math.round(points[0]??.x ?? 0),
    y: Math.round(points[0]??.y ?? 0),
    t: 0
  };
  const end = path[path.length - 1] ?? start;
  const beforeScore = this.score;
  const beforeEnergy = this.energy;
  const hit = path.length >= 2 ? this.findTetherOnPath(path) : undefined;
  const target: SwipeEvent['target'] = hit ? 'tether' : 'empty';
  const message: SwipeEvent['message'] = hit ? (hit.distanceSq < 30 * 30 ? 'PERFECT' : 'HARVEST') : 'MISS';
  if (hit) {
    this.captureOrb(hit.orb);
  }

  const event: SwipeEvent = {
    t: Math.round(this.timeMs),
    x1: start.x,
    y1: start.y,
    x2: end.x,
    y2: end.y,
    target,
    success: Boolean(hit),
    orbId: hit?.orb.id,
    scoreDelta: this.score - beforeScore,
    energyDelta: Math.round((this.energy - beforeEnergy) * 10) / 10,
    message,
    path
  };
  this.swipeEvents.push(event);
  return event;
}

applySwipe(x1: number, y1: number, x2: number, y2: number): SwipeEvent {
  return this.applySwipePath([
    { x: x1, y: y1, t: 0 },
    { x: x2, y: y2, t: 16 }
  ]);
}

forceEnd(): void {
  this.energy = 0;
  this.endRun();
}

getSnapshot(): SimulationSnapshot {
  return {
    timeMs: Math.round(this.timeMs),
    score: this.score,
    energy: this.energy,
    energyCaptured: this.energyCaptured,
    streak: this.streak,
    maxStreak: this.maxStreak,
    harvestCount: this.harvestCount,
    phase: this.phase,
    ended: this.ended,
    collapseT: clamp(this.collapseMs / 2200, 0, 1),
    orbs: this.orbs,
    flybys: this.flybys,
    shadowArms: this.shadowArms
  };
}

getReplayPayload(): ReplayPayload {
  return {
    version: 1,
    seed: this.seed,
    startedAt: this.startedAt,
    survivalMs: Math.round(this.ended ? this.timeMs - this.collapseMs : this.timeMs),
    score: this.score,
    energyCaptured: this.energyCaptured,
    maxStreak: this.maxStreak,
    tapEvents: this.tapEvents.map((event) => ({ ...event })),
    swipeEvents: this.swipeEvents.map((event) => ({ ...event })),
    phaseTransitions: this.phaseTransitions.map((event) => ({ ...event }))
  };
}

runInputs(inputs: TimedInput[], untilMs: number): ReplayPayload {
  let inputIndex = 0;
  const ordered = [...inputs].sort((a, b) => a.t - b.t);
  while (this.timeMs < untilMs && !this.ended) {
    while (inputIndex < ordered.length && ordered[inputIndex].t <= this.timeMs) {
      const input = ordered[inputIndex];
      if (input.kind === 'tap') {
        this.applyTap(input.x, input.y);
      } else if (input.path) {

```

```

        this.applySwipePath(input.path);
    } else {
        this.applySwipe(input.x, input.y, input.x2 ?? input.x, input.y2 ?? input.y);
    }
    inputIndex += 1;
}
this.step(1000 / 60);
}
return this.getReplayPayload();
}

private initPools(): void {
    this.orbs.length = 0;
    this.flybys.length = 0;
    this.shadowArms.length = 0;
    for (let index = 0; index < ORB_POOL_SIZE; index += 1) {
        this.orbs.push(this.makeInactiveOrb(index));
    }
    for (let index = 0; index < FLYBY_POOL_SIZE; index += 1) {
        this.flybys.push(this.makeInactiveFlyby(index));
    }
    for (let index = 0; index < 3; index += 1) {
        this.shadowArms.push({
            angle: index * ((Math.PI * 2) / 3),
            stunMs: 0,
            intensity: 0
        });
    }
}

private makeInactiveOrb(index: number): OrbState {
    return {
        active: false,
        captured: false,
        tutorial: false,
        id: -index,
        x: 0,
        y: 0,
        vx: 0,
        vy: 0,
        radius: 26,
        ageMs: 0,
        frozenMs: 0,
        energy: 1,
        tetherPhase: 0
    };
}

private makeInactiveFlyby(index: number): FlybyState {
    return {
        active: false,
        id: -index,
        x: 0,
        y: 0,
        vx: 0,
        vy: 0,
        radius: 24,
        ageMs: 0
    };
}

private spawnOrbs(dtMs: number): void {
    this.orbSpawnMs -= dtMs;
    if (this.orbSpawnMs > 0) {
        return;
    }
    const tutorialSpawn = this.timeMs < 16000 && this.nextOrbId <= 5;
    this.orbSpawnMs = tutorialSpawn ? 2300 : clamp(1020 - this.phase * 120 - this.timeMs * 0.006, 430, 1100);
    const orb = this.orbs.find((candidate) => !candidate.active);
    if (!orb) {
        return;
    }

    const preset = tutorialSpawn ? this.getTutorialOrbPreset(this.nextOrbId) : undefined;
    const edge = Math.floor(this.rng() * 4);
    const margin = 96;
    let x = preset?.x ?? margin + this.rng() * (WORLD_WIDTH - margin * 2);
    let y = preset?.y ?? margin + this.rng() * (WORLD_HEIGHT - margin * 2);
    if (!preset) {
        if (edge === 0) {
            y = margin;
        } else if (edge === 1) {
            x = WORLD_WIDTH - margin;
        } else if (edge === 2) {
            y = WORLD_HEIGHT - margin * 1.9;
        } else {
            x = margin;
        }
    }

    const angle = Math.atan2(y - BLACK_HOLE_Y, x - BLACK_HOLE_X);
    const tangentSign = this.rng() > 0.5 ? 1 : -1;
    const tangent = angle + (Math.PI / 2) * tangentSign + (this.rng() - 0.5) * 0.45;
    const speed = preset?.speed ?? 76 + this.rng() * 48 + this.phase * 12;
    const inwardX = BLACK_HOLE_X - x;
    const inwardY = BLACK_HOLE_Y - y;
    const inwardLength = Math.max(1, Math.hypot(inwardX, inwardY));

    orb.active = true;
    orb.captured = false;
    orb.tutorial = tutorialSpawn;
    orb.id = this.nextOrbId;
    this.nextOrbId += 1;
    orb.x = x;
    orb.y = y;
    orb.vx = preset?.vx ?? Math.cos(tangent) * speed + (inwardX / inwardLength) * 42;
    orb.vy = preset?.vy ?? Math.sin(tangent) * speed + (inwardY / inwardLength) * 42;
    orb.radius = preset?.radius ?? 24 + this.rng() * 13;
    orb.ageMs = 0;
    orb.frozenMs = 0;
    orb.energy = preset?.energy ?? 2 + this.rng() * 2.8;
    orb.tetherPhase = this.rng() * Math.PI * 2;
}

private getTutorialOrbPreset(id: number): { x: number; y: number; vx: number; vy: number; radius: number; energy: number; speed: number } {
    const presets = [
        { x: 825, y: 1055, vx: -18, vy: 8, radius: 42, energy: 5.2, speed: 38 },
        { x: 265, y: 1090, vx: 20, vy: 4, radius: 30, energy: 4.7, speed: 42 },
        { x: 790, y: 1280, vx: -22, vy: -8, radius: 38, energy: 4.3, speed: 48 },
        { x: 318, y: 1320, vx: 24, vy: -12, radius: 36, energy: 4, speed: 52 },
    ];
    return presets[id % presets.length];
}

```

```

    { x: 730, y: 720, vx: -14, vy: 18, radius: 36, energy: 3.9, speed: 54 }
  ];
  return presets[(id - 1) % presets.length];
}

private spawnFlybys(dtMs: number): void {
  if (this.harvestCount === 0 && this.timeMs < 20000) {
    this.flybySpawnMs = 900;
    return;
  }
  this.flybySpawnMs -= dtMs;
  if (this.flybySpawnMs > 0) {
    return;
  }
  this.flybySpawnMs = 4700 + this.rng() * 1900 - this.phase * 320;
  const flyby = this.flybys.find((candidate) => !candidate.active);
  if (!flyby) {
    return;
  }
  const fromLeft = this.rng() > 0.5;
  const y = 240 + this.rng() * (WORLD_HEIGHT - 620);
  const speed = 760 + this.rng() * 240;
  flyby.active = true;
  flyby.id = this.nextFlybyId;
  this.nextFlybyId += 1;
  flyby.x = fromLeft ? -80 : WORLD_WIDTH + 80;
  flyby.y = y;
  flyby.vx = fromLeft ? speed : -speed;
  flyby.vy = (this.rng() - 0.5) * 110;
  flyby.radius = 22 + this.rng() * 9;
  flyby.ageMs = 0;
}

private updateShadowArms(dtMs: number): void {
  const dt = dtMs / 1000;
  const onboardingQuiet = this.harvestCount === 0 && this.timeMs < 20000;
  for (let index = 0; index < this.shadowArms.length; index += 1) {
    const arm = this.shadowArms[index];
    arm.angle += dt * (0.12 + this.phase * 0.045) * (index % 2 === 0 ? 1 : -1);
    arm.stunMs = Math.max(0, arm.stunMs - dtMs);
    arm.intensity = onboardingQuiet ? 0 : arm.stunMs > 0 ? 0.25 : clamp((this.phase - 1) / 3, 0, 1);
  }
}

private updateOrbs(dtMs: number, dt: number): void {
  for (const orb of this.orbs) {
    if (!orb.active) {
      continue;
    }

    orb.ageMs += dtMs;
    if (orb.captured) {
      const captureT = clamp(dt * 8, 0, 1);
      orb.x += (BLACK_HOLE_X - orb.x) * captureT;
      orb.y += (WORLD_HEIGHT - 112 - orb.y) * captureT;
      orb.radius *= 1 + dt * 0.8;
      if (orb.ageMs > 460) {
        orb.active = false;
      }
      continue;
    }

    if (orb.frozenMs > 0) {
      orb.frozenMs = Math.max(0, orb.frozenMs - dtMs);
    } else {
      const dx = BLACK_HOLE_X - orb.x;
      const dy = BLACK_HOLE_Y - orb.y;
      const radiusSq = dx * dx + dy * dy;
      const radius = Math.max(48, Math.sqrt(radiusSq));
      const gravity = (this.phase >= 3 ? 420000 : 190000) / radiusSq;
      orb.vx += (dx / radius) * gravity * dt;
      orb.vy += (dy / radius) * gravity * dt;
      this.applyArmInfluence(orb, dt);
      orb.x += orb.vx * dt;
      orb.y += orb.vy * dt;
    }

    const distanceToHole = distanceSquared(orb.x, orb.y, BLACK_HOLE_X, BLACK_HOLE_Y);
    if (distanceToHole < EVENT_HORIZON_RADIUS * EVENT_HORIZON_RADIUS) {
      this.missOrb(orb);
    } else if (
      orb.x < -240 ||
      orb.x > WORLD_WIDTH + 240 ||
      orb.y < -260 ||
      orb.y > WORLD_HEIGHT + 260 ||
      orb.ageMs > 17500
    ) {
      this.missOrb(orb);
    }
  }
}

private updateFlybys(dtMs: number, dt: number): void {
  for (const flyby of this.flybys) {
    if (!flyby.active) {
      continue;
    }

    flyby.ageMs += dtMs;
    flyby.x += flyby.vx * dt;
    flyby.y += flyby.vy * dt;
    if (
      flyby.x < -180 ||
      flyby.x > WORLD_WIDTH + 180 ||
      flyby.y < -160 ||
      flyby.y > WORLD_HEIGHT + 160 ||
      flyby.ageMs > 4500
    ) {
      flyby.active = false;
    }
  }
}

private applyArmInfluence(orb: OrbState, dt: number): void {
  if (this.phase < 2) {
    return;
  }
  for (const arm of this.shadowArms) {
    if (arm.stunMs > 0) {
      continue;
    }
  }
}

```

```

    }
    const endX = BLACK_HOLE_X + Math.cos(arm.angle) * ARM_LENGTH;
    const endY = BLACK_HOLE_Y + Math.sin(arm.angle) * ARM_LENGTH;
    const distance = distanceToSegmentSquared(orb.x, orb.y, BLACK_HOLE_X, BLACK_HOLE_Y, endX, endY);
    if (distance > (ARM_HIT_WIDTH + orb.radius) * (ARM_HIT_WIDTH + orb.radius)) {
        continue;
    }
    const normalX = Math.cos(arm.angle + Math.PI / 2);
    const normalY = Math.sin(arm.angle + Math.PI / 2);
    const push = this.phase >= 4 ? -130 : 92;
    orb.vx += normalX * push * dt;
    orb.vy += normalY * push * dt;
    if (this.phase >= 4) {
        orb.vx += (BLACK_HOLE_X - orb.x) * 0.17 * dt;
        orb.vy += (BLACK_HOLE_Y - orb.y) * 0.17 * dt;
    }
}
}

private updatePhase(): void {
    const nextPhase: GamePhase =
        this.energy > 64 ? 1 : this.energy > 39 ? 2 : this.energy > 18 ? 3 : 4;
    if (nextPhase !== this.phase) {
        this.phase = nextPhase;
        this.phaseTransitions.push({
            t: Math.round(this.timeMs),
            phase: this.phase,
            energy: Math.round(this.energy * 10) / 10
        });
    }
}

private findOrb(x: number, y: number, radius: number): OrbState | undefined {
    let best: OrbState | undefined;
    let bestDistance = Number.POSITIVE_INFINITY;
    for (const orb of this.orbs) {
        if (!orb.active || orb.captured) {
            continue;
        }
        const hitRadius = radius + orb.radius;
        const dist = distanceSquared(x, y, orb.x, orb.y);
        if (dist < hitRadius * hitRadius && dist < bestDistance) {
            best = orb;
            bestDistance = dist;
        }
    }
    return best;
}

private findTetherOnPath(path: readonly GesturePathPoint[]): TetherHit | undefined {
    let best: TetherHit | undefined;
    let bestScore = Number.POSITIVE_INFINITY;
    for (const orb of this.orbs) {
        if (!orb.active || orb.captured) {
            continue;
        }
        const tetherX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * TETHER_OUTER_START;
        const tetherY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * TETHER_OUTER_START;
        const hitWidth = TETHER_HIT_WIDTH + (orb.tutorial ? 10 : 0);
        const hitWidthSq = hitWidth * hitWidth;
        let closest = Number.POSITIVE_INFINITY;
        for (let index = 1; index < path.length; index += 1) {
            const previous = path[index - 1];
            const point = path[index];
            const dist = segmentToSegmentDistanceSquared(
                previous.x,
                previous.y,
                point.x,
                point.y,
                tetherX,
                tetherY,
                orb.x,
                orb.y
            );
            if (dist < closest) {
                closest = dist;
            }
        }
        if (closest <= hitWidthSq) {
            const priorityBonus = (orb.tutorial ? 1200 : 0) + (orb.frozenMs > 0 ? 600 : 0) + orb.energy * 12;
            const score = closest - priorityBonus;
            if (score < bestScore) {
                best = { orb, distanceSq: closest };
                bestScore = score;
            }
        }
    }
    return best;
}

private findFlyby(x: number, y: number): FlybyState | undefined {
    for (const flyby of this.flybys) {
        if (!flyby.active) {
            continue;
        }
        const hitRadius = flyby.radius + FLYBY_CAPTURE_RADIUS;
        if (distanceSquared(x, y, flyby.x, flyby.y) <= hitRadius * hitRadius) {
            return flyby;
        }
    }
    return undefined;
}

private stunArm(x: number, y: number): boolean {
    if (this.phase < 2) {
        return false;
    }
    for (const arm of this.shadowArms) {
        const endX = BLACK_HOLE_X + Math.cos(arm.angle) * ARM_LENGTH;
        const endY = BLACK_HOLE_Y + Math.sin(arm.angle) * ARM_LENGTH;
        if (distanceToSegmentSquared(x, y, BLACK_HOLE_X, BLACK_HOLE_Y, endX, endY) < 76 * 76) {
            arm.stunMs = 1050;
            this.score += 16;
            return true;
        }
    }
    return false;
}
}

```



```

private captureOrb(orb: OrbState): void {
  orb.captured = true;
  orb.tutorial = false;
  orb.ageMs = 0;
  orb.frozenMs = 0;
  orb.vx = 0;
  orb.vy = 0;
  const energyGain = orb.energy + 2.1;
  this.energy = clamp(this.energy + energyGain, 0, MAX_ENERGY);
  this.energyCaptured += Math.round(energyGain);
  this.streak += 1;
  this.harvestCount += 1;
  this.maxStreak = Math.max(this.maxStreak, this.streak);
  this.score += Math.round(72 + energyGain * 24 + this.streak * 7 + this.phase * 13);
}

private collectFlyby(flyby: FlybyState): void {
  flyby.active = false;
  this.energy = clamp(this.energy + 1.4, 0, MAX_ENERGY);
  this.energyCaptured += 1;
  this.streak += 1;
  this.maxStreak = Math.max(this.maxStreak, this.streak);
  this.score += 115 + this.phase * 18 + this.streak * 3;
}

private missOrb(orb: OrbState): void {
  orb.active = false;
  this.streak = 0;
  const onboardingGrace = this.timeMs < 30000 || this.harvestCount === 0;
  const penalty = onboardingGrace ? 2.2 + this.phase * 0.35 : 6.5 + this.phase * 1.2;
  this.energy = clamp(this.energy - penalty, 0, MAX_ENERGY);
}

private endRun(): void {
  if (this.ended) {
    return;
  }
  this.ended = true;
  this.collapseMs = 0;
  this.energy = 0;
}
}

```

src/game/gestures.ts

```

import { distanceSquared, distanceToSegmentSquared } from './math';

export interface WorldPoint {
  x: number;
  y: number;
}

export interface GesturePoint extends WorldPoint {
  screenX: number;
  screenY: number;
  t: number;
}

export interface SwipeGesture {
  points: GesturePoint[];
  start: GesturePoint;
  end: GesturePoint;
  screenDistance: number;
  worldDistance: number;
  durationMs: number;
}

export interface RecordedGesturePoint extends WorldPoint {
  t: number;
}

export type GesturePathPoint = WorldPoint & { t?: number };

export function screenPathLength(points: readonly GesturePoint[]): number {
  let distance = 0;
  for (let index = 1; index < points.length; index += 1) {
    const previous = points[index - 1];
    const point = points[index];
    distance += Math.hypot(point.screenX - previous.screenX, point.screenY - previous.screenY);
  }
  return distance;
}

export function worldPathLength(points: readonly WorldPoint[]): number {
  let distance = 0;
  for (let index = 1; index < points.length; index += 1) {
    const previous = points[index - 1];
    const point = points[index];
    distance += Math.hypot(point.x - previous.x, point.y - previous.y);
  }
  return distance;
}

export function makeSwipeGesture(points: readonly GesturePoint[]): SwipeGesture {
  const copied = points.map((point) => ({ ...point }));
  const start = copied[0];
  const end = copied[copied.length - 1];
  return {
    points: copied,
    start,
    end,
    screenDistance: screenPathLength(copied),
    worldDistance: worldPathLength(copied),
    durationMs: Math.max(0, end.t - start.t)
  };
}

export function quantizeGesturePath(
  points: readonly GesturePathPoint[],
  maxPoints = 24
): RecordedGesturePoint[] {
  if (points.length === 0) {
    return [];
  }
  const simplified = simplifyWorldPath(points, maxPoints);

```

```

const baseT = simplified[0].t ?? points[0].t ?? 0;
return simplified.map((point, index) => ({
  x: Math.round(point.x),
  y: Math.round(point.y),
  t: Math.max(0, Math.round((point.t ?? baseT + index * 16) - baseT))
})));
}

export function simplifyWorldPath<T extends WorldPoint>(points: readonly T[], maxPoints = 24): T[] {
  if (points.length <= maxPoints) {
    return points.map((point) => point);
  }

  let toleranceSq = 12 * 12;
  let simplified = rdp(points, toleranceSq);
  while (simplified.length > maxPoints && toleranceSq < 180 * 180) {
    toleranceSq *= 1.65;
    simplified = rdp(points, toleranceSq);
  }

  if (simplified.length <= maxPoints) {
    return simplified;
  }

  const sampled: T[] = [];
  for (let index = 0; index < maxPoints; index += 1) {
    const sourceIndex = Math.round((index / (maxPoints - 1)) * (simplified.length - 1));
    sampled.push(simplified[sourceIndex]);
  }
  return sampled;
}

function rdp<T extends WorldPoint>(points: readonly T[], toleranceSq: number): T[] {
  if (points.length <= 2) {
    return points.map((point) => point);
  }

  const keep = new Array<boolean>(points.length).fill(false);
  keep[0] = true;
  keep[points.length - 1] = true;
  simplifyRange(points, keep, 0, points.length - 1, toleranceSq);
  return points.filter((_, index) => keep[index]);
}

function simplifyRange<T extends WorldPoint>({
  points: readonly T[],
  keep: boolean[],
  startIndex: number,
  endIndex: number,
  toleranceSq: number
}): void {
  let bestIndex = -1;
  let bestDistance = 0;
  const start = points[startIndex];
  const end = points[endIndex];

  for (let index = startIndex + 1; index < endIndex; index += 1) {
    const point = points[index];
    const distance = distanceToSegmentSquared(point.x, point.y, start.x, start.y, end.x, end.y);
    if (distance > bestDistance) {
      bestDistance = distance;
      bestIndex = index;
    }
  }

  if (bestIndex === -1 || bestDistance <= toleranceSq) {
    return;
  }

  keep[bestIndex] = true;
  simplifyRange(points, keep, startIndex, bestIndex, toleranceSq);
  simplifyRange(points, keep, bestIndex, endIndex, toleranceSq);
}

export function segmentToSegmentDistanceSquared(
  ax: number,
  ay: number,
  bx: number,
  by: number,
  cx: number,
  cy: number,
  dx: number,
  dy: number
): number {
  if (segmentsIntersect(ax, ay, bx, by, cx, cy, dx, dy)) {
    return 0;
  }
  return Math.min(
    distanceToSegmentSquared(ax, ay, cx, cy, dx, dy),
    distanceToSegmentSquared(bx, by, cx, cy, dx, dy),
    distanceToSegmentSquared(cx, cy, ax, ay, bx, by),
    distanceToSegmentSquared(dx, dy, ax, ay, bx, by)
  );
}

function segmentsIntersect(
  ax: number,
  ay: number,
  bx: number,
  by: number,
  cx: number,
  cy: number,
  dx: number,
  dy: number
): boolean {
  const o1 = orientation(ax, ay, bx, by, cx, cy);
  const o2 = orientation(ax, ay, bx, by, dx, dy);
  const o3 = orientation(cx, cy, dx, dy, ax, ay);
  const o4 = orientation(cx, cy, dx, dy, bx, by);

  if (o1 !== o2 && o3 !== o4) {
    return true;
  }

  return (
    (o1 === 0 && onSegment(ax, ay, cx, cy, bx, by)) ||
    (o2 === 0 && onSegment(ax, ay, dx, dy, bx, by)) ||
    (o3 === 0 && onSegment(cx, cy, ax, ay, dx, dy)) ||
    (o4 === 0 && onSegment(cx, cy, bx, by, dx, dy))
  );
}

```

```

    });
}

function orientation(ax: number, ay: number, bx: number, by: number, cx: number, cy: number): -1 | 0 | 1 {
    const value = (by - ay) * (cx - bx) - (bx - ax) * (cy - by);
    if (Math.abs(value) < 0.000001) {
        return 0;
    }
    return value > 0 ? 1 : -1;
}

function onSegment(ax: number, ay: number, px: number, py: number, bx: number, by: number): boolean {
    return (
        px <= Math.max(ax, bx) &&
        px >= Math.min(ax, bx) &&
        py <= Math.max(ay, by) &&
        py >= Math.min(ay, by) &&
        distanceSquared(px, py, ax, ay) + distanceSquared(px, py, bx, by) >= distanceSquared(ax, ay, bx, by) - 0.01
    );
}

```

src/game/types.ts

```

export type GamePhase = 1 | 2 | 3 | 4;

export interface TapEvent {
    t: number;
    x: number;
    y: number;
    target: 'orb' | 'flyby' | 'arm' | 'empty';
}

export interface RecordedGesturePoint {
    x: number;
    y: number;
    t: number;
}

export type SwipeTarget = 'orb' | 'tether' | 'arm' | 'flyby' | 'empty';
export type SwipeMessage = 'HARVEST' | 'PERFECT' | 'MISS' | 'CANCELED';

export interface SwipeEvent {
    t: number;
    x1: number;
    y1: number;
    x2: number;
    y2: number;
    target: SwipeTarget;
    success: boolean;
    orbId?: number;
    scoreDelta?: number;
    energyDelta?: number;
    message: SwipeMessage;
    path: RecordedGesturePoint[];
}

export interface PhaseTransition {
    t: number;
    phase: GamePhase;
    energy: number;
}

export interface ReplayPayload {
    version: 1;
    seed: string;
    startedAt: number;
    survivalMs: number;
    score: number;
    energyCaptured: number;
    maxStreak: number;
    tapEvents: TapEvent[];
    swipeEvents: SwipeEvent[];
    phaseTransitions: PhaseTransition[];
}

export interface OrbState {
    active: boolean;
    captured: boolean;
    tutorial: boolean;
    id: number;
    x: number;
    y: number;
    vx: number;
    vy: number;
    radius: number;
    ageMs: number;
    frozenMs: number;
    energy: number;
    tetherPhase: number;
}

export interface FlybyState {
    active: boolean;
    id: number;
    x: number;
    y: number;
    vx: number;
    vy: number;
    radius: number;
    ageMs: number;
}

export interface ShadowArmState {
    angle: number;
    stunMs: number;
    intensity: number;
}

export interface SimulationSnapshot {
    timeMs: number;
    score: number;
    energy: number;
    energyCaptured: number;
    streak: number;
    maxStreak: number;
    harvestCount: number;
}

```

```

    phase: GamePhase;
    ended: boolean;
    collapseT: number;
    orbs: readonly OrbState[];
    flybys: readonly FlybyState[];
    shadowArms: readonly ShadowArmState[];
  }

export interface PosterStats {
  score: number;
  survivalMs: number;
  seed: string;
  phase: GamePhase;
}

```

src/main.ts

```

import './styles.css';
import { EventHorizonGame } from './game/EventHorizonGame';

const root = document.querySelector<HTMLDivElement>('#game-root');
const restartButton = document.querySelector<HTMLButtonElement>('#restart-button');
const shareButton = document.querySelector<HTMLButtonElement>('#share-button');
const helpButton = document.querySelector<HTMLButtonElement>('#help-button');
const helpOverlay = document.querySelector<HTMLDivElement>('#help-overlay');
const helpPlayButton = document.querySelector<HTMLButtonElement>('#help-play-button');
const posterLink = document.querySelector<HTMLAnchorElement>('#poster-link');

if (!root || !restartButton || !shareButton || !helpButton || !helpOverlay || !helpPlayButton || !posterLink) {
  throw new Error('Event Horizon shell is missing required DOM nodes.');
```

```

}

const game = new EventHorizonGame(root, {
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000
});

await game.start();

const debugInput = new URLSearchParams(window.location.search).get('debugInput') === '1';
game.setInputDebug(debugInput);

const hasSeenHelp = (): boolean => {
  try {
    return localStorage.getItem('eventHorizon.helpSeen') === '1';
  } catch {
    return false;
  }
};

const markHelpSeen = (): void => {
  try {
    localStorage.setItem('eventHorizon.helpSeen', '1');
  } catch {
    // localStorage can be unavailable in locked-down browser modes.
  }
};

const openHelp = (): void => {
  helpOverlay.hidden = false;
  helpPlayButton.textContent = game.getSnapshot().timeMs > 0 ? 'RESUME' : 'PLAY';
  game.setPaused(true);
};

const closeHelp = (): void => {
  helpOverlay.hidden = true;
  markHelpSeen();
  game.setPaused(false);
};

if (!hasSeenHelp()) {
  openHelp();
}

restartButton.addEventListener('click', () => {
  posterLink.removeAttribute('href');
  game.restart();
});

helpButton.addEventListener('click', openHelp);
helpPlayButton.addEventListener('click', closeHelp);

shareButton.addEventListener('click', async () => {
  const poster = await game.exportPoster();
  posterLink.href = poster;
});

declare global {
  interface Window {
    _EVENT_HORIZON_?: {
      exportPoster: () => Promise<string>;
      forceEnd: () => void;
      getReplayPayload: () => ReturnType<EventHorizonGame['getReplayPayload']>;
      getSnapshot: () => ReturnType<EventHorizonGame['getSnapshot']>;
      restart: () => void;
    };
    _EVENT_HORIZON_DEBUG_?: {
      forceHelp: (open: boolean) => void;
      getLastGesture: () => ReturnType<EventHorizonGame['getLastGesture']>;
      getReplayPayload: () => ReturnType<EventHorizonGame['getReplayPayload']>;
      getSnapshot: () => ReturnType<EventHorizonGame['getSnapshot']>;
      setInputDebug: (enabled: boolean) => void;
      simulateSwipeWorld: (points: { x: number; y: number }[]) => ReturnType<EventHorizonGame['simulateSwipeWorld']>;
      simulateTapWorld: (x: number, y: number) => ReturnType<EventHorizonGame['simulateTapWorld']>;
    };
  }
}

window._EVENT_HORIZON_ = {
  exportPoster: () => game.exportPoster(),
  forceEnd: () => game.forceEnd(),
  getReplayPayload: () => game.getReplayPayload(),
  getSnapshot: () => game.getSnapshot(),
  restart: () => game.restart()
};

```

```

window.__EVENT_HORIZON_DEBUG__ = {
  forceHelp: (open) => {
    if (open) {
      openHelp();
    } else {
      closeHelp();
    }
  },
  getLastGesture: () => game.getLastGesture(),
  getReplayPayload: () => game.getReplayPayload(),
  getSnapshot: () => game.getSnapshot(),
  setInputDebug: (enabled) => game.setInputDebug(enabled),
  simulateSwipeWorld: (points) => game.simulateSwipeWorld(points),
  simulateTapWorld: (x, y) => game.simulateTapWorld(x, y)
};

```

src/styles.css

```

:root {
  color-scheme: dark;
  font-family:
    Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  background: #03040a;
  color: #f7fbff;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
}

* {
  box-sizing: border-box;
  -webkit-tap-highlight-color: transparent;
}

html,
body,
#app,
#game-shell,
#game-root {
  width: 100%;
  height: 100%;
  margin: 0;
  overflow: hidden;
  overscroll-behavior: none;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

body {
  position: fixed;
  inset: 0;
  background:
    radial-gradient(circle at 50% 42%, rgba(22, 49, 89, 0.24), transparent 38%),
    linear-gradient(180deg, #070912 0%, #03040a 100%);
}

button,
a {
  -webkit-tap-highlight-color: transparent;
}

#game-shell {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  background: #03040a;
}

#game-root {
  position: absolute;
  inset: 0;
  overflow: hidden;
}

#game-root canvas {
  display: block;
  width: 100%;
  height: 100%;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

#status-layer {
  position: fixed;
  top: max(12px, env(safe-area-inset-top));
  right: max(12px, env(safe-area-inset-right));
  z-index: 2;
  display: flex;
  gap: 8px;
}

#restart-button,
#share-button,
#help-button,
#poster-link {
  display: grid;
  width: 40px;
  height: 40px;
  place-items: center;
  border: 1px solid rgba(179, 226, 255, 0.26);
  border-radius: 8px;
  background: rgba(7, 12, 25, 0.72);
  color: #f7fbff;
  font-size: 21px;
  line-height: 1;
  text-decoration: none;
  backdrop-filter: blur(8px);
  touch-action: manipulation;
}

#poster-link:not([href]) {

```

```

display: none;
}

#help-overlay {
position: fixed;
inset: 0;
z-index: 5;
display: grid;
place-items: center;
padding: max(18px, env(safe-area-inset-top)) max(16px, env(safe-area-inset-right))
  max(18px, env(safe-area-inset-bottom)) max(16px, env(safe-area-inset-left));
background: rgba(2, 4, 10, 0.82);
backdrop-filter: blur(12px);
touch-action: none;
}

#help-overlay[hidden] {
display: none;
}

#help-panel {
width: min(92vw, 440px);
max-height: min(88vh, 760px);
overflow: auto;
padding: 22px;
border: 1px solid rgba(139, 222, 255, 0.34);
border-radius: 8px;
background: rgba(7, 12, 25, 0.94);
box-shadow: 0 20px 80px rgba(0, 0, 0, 0.48);
color: #f7fbff;
}

#help-panel h1 {
margin: 12px 0 8px;
font-size: 28px;
letter-spacing: 0;
}

#help-panel p {
margin: 10px 0;
color: #cfefff;
font-size: 16px;
line-height: 1.45;
}

#help-panel ol {
margin: 14px 0;
padding-left: 22px;
color: #f2fbff;
font-size: 15px;
line-height: 1.45;
}

#help-panel li {
margin: 8px 0;
}

#help-play-button {
width: 100%;
min-height: 52px;
margin-top: 12px;
border: 0;
border-radius: 8px;
background: linear-gradient(90deg, #67f4ff, #d267ff);
color: #061120;
font-size: 17px;
font-weight: 900;
letter-spacing: 0;
touch-action: manipulation;
}

.help-example {
position: relative;
height: 126px;
margin: 2px 0 10px;
overflow: hidden;
border-radius: 8px;
background:
  radial-gradient(circle at 50% 44%, rgba(82, 28, 120, 0.28), transparent 34%),
  radial-gradient(circle at 72% 58%, rgba(103, 244, 255, 0.14), transparent 18%),
  #03040a;
}

.help-hole,
.help-orb,
.help-tether,
.help-swipe {
position: absolute;
display: block;
}

.help-hole {
left: 44%;
top: 38%;
width: 48px;
height: 48px;
border: 5px solid rgba(103, 244, 255, 0.34);
border-radius: 50%;
background: #000;
box-shadow: 0 0 34px rgba(103, 244, 255, 0.22);
}

.help-orb {
left: 70%;
top: 58%;
width: 30px;
height: 30px;
border-radius: 50%;
background: #d9fbff;
box-shadow: 0 0 20px #67f4ff, 0 0 46px rgba(210, 103, 255, 0.7);
}

.help-tether {
left: 52%;
top: 54%;
width: 105px;
height: 4px;
border-radius: 999px;
background: #67f4ff;
}

```

```

    box-shadow: 0 0 18px #67f4ff;
    transform: rotate(18deg);
    transform-origin: left center;
  }

.help-swipe {
  left: 58%;
  top: 42%;
  width: 104px;
  height: 42px;
  border-bottom: 7px solid #ffffff;
  border-left: 7px solid transparent;
  border-radius: 0 0 80px 80px;
  box-shadow: 0 12px 20px rgba(210, 103, 255, 0.5);
  transform: rotate(-22deg);
}

@media (min-width: 820px) {
  #game-root {
    inset: 0;
  }
}

```

tests/e2e/playable.spec.ts

```

import { expect, test, type Page } from '@playwright/test';

const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;
const BLACK_HOLE_X = WORLD_WIDTH / 2;
const BLACK_HOLE_Y = WORLD_HEIGHT * 0.44;

test('help overlay opens on first visit and closes with play', async ({ page }) => {
  await openGame(page);
  await expect(page.locator('#help-overlay')).toBeVisible();
  await page.locator('#help-play-button').click();
  await expect(page.locator('#help-overlay')).toBeHidden();
  await page.waitForTimeout(250);
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot());
  expect(snapshot?.timeMs).toBeGreaterThan(0);
});

test('smoke renders playable Pixi canvas', async ({ page }) => {
  await openGameAndPlay(page);
  const canvas = page.locator('canvas');
  await expect(canvas).toBeVisible();
  await page.waitForTimeout(650);
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot());
  expect(snapshot?.phase).toBeGreaterThanOrEqual(1);
  expect(snapshot?.energy).toBeGreaterThan(0);
});

test('tap and tether swipe are captured into replay payload', async ({ page }) => {
  await openGameAndPlay(page, 'debugInput=1');
  const canvas = page.locator('canvas');
  const box = await canvas.boundingBox();
  expect(box).toBeTruthy();
  if (!box) {
    return;
  }

  await page.touchscreen.tap(box.x + box.width * 0.5, box.y + box.height * 0.5);
  await waitForTutorial0rb(page);
  const before = await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot());
  const swipe = await makeTutorialTetherSwipe(page);
  await page.mouse.move(swipe.start.x, swipe.start.y);
  await page.mouse.down();
  await page.mouse.move(swipe.mid.x, swipe.mid.y, { steps: 4 });
  await page.mouse.move(swipe.end.x, swipe.end.y, { steps: 4 });
  await page.mouse.up();
  await page.waitForTimeout(120);

  const replay = await page.evaluate(() => window.__EVENT_HORIZON__.getReplayPayload());
  const after = await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot());
  const lastGesture = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__.getLastGesture());
  const lastSwipe = replay?.swipeEvents.at(-1);
  expect(replay?.tapEvents.length).toBeGreaterThanOrEqual(1);
  expect(replay?.swipeEvents.length).toBeGreaterThanOrEqual(1);
  expect(lastSwipe?.path.length).toBeGreaterThanOrEqual(2);
  expect(lastSwipe?.target).not.toBe('empty');
  expect(lastSwipe?.success).toBe(true);
  expect(after?.score).toBeGreaterThan(before?.score ?? 0);
  expect(lastGesture?.activeTrailCount).toBeGreaterThan(0);
});

test('posterizer exports a compact png data url', async ({ page }) => {
  await openGameAndPlay(page);
  await page.waitForTimeout(900);
  const poster = await page.evaluate(() => window.__EVENT_HORIZON__.exportPoster());
  expect(poster).toMatch(/^data:image\/png;base64,/);
  expect(poster?.length).toBeGreaterThan(1200);
});

test('miss swipe records visible feedback state', async ({ page }) => {
  await openGameAndPlay(page, 'debugInput=1');
  await page.waitForTimeout(250);
  const result = await page.evaluate(() =>
    window.__EVENT_HORIZON_DEBUG__.simulateSwipeWorld([
      { x: 80, y: 1760 },
      { x: 180, y: 1840 }
    ])
  );
  const lastGesture = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__.getLastGesture());
  expect(result?.success).toBe(false);
  expect(result?.message).toBe('MISS');
  expect(lastGesture?.activeTrailCount).toBeGreaterThan(0);
});

test('end-state collapse is reachable', async ({ page }) => {
  await openGameAndPlay(page);
  await page.evaluate(() => window.__EVENT_HORIZON__.forceEnd());
  await page.waitForTimeout(250);
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot());
  expect(snapshot?.ended).toBe(true);
  expect(snapshot?.collapseT).toBeGreaterThan(0);
});

```

```

async function openGame(page: Page, query = ''): Promise<void> {
  await page.addInitScript(() => {
    localStorage.clear();
  });
  await page.goto(query ? `.${query}` : './');
  await page.locator('canvas').waitFor({ state: 'visible' });
}

async function openGameAndPlay(page: Page, query = ''): Promise<void> {
  await openGame(page, query);
  await expect(page.locator('#help-overlay')).toBeVisible();
  await page.locator('#help-play-button').click();
  await expect(page.locator('#help-overlay')).toBeHidden();
}

async function waitForTutorialOrb(page: Page) {
  return page.waitForFunction(() => {
    const snapshot = window.__EVENT_HORIZON__?.getSnapshot();
    return snapshot?.orbs.some((orb) => orb.active && orb.tutorial && !orb.captured);
  });
}

async function makeTutorialTetherSwipe(page: Page) {
  const canvas = page.locator('canvas');
  const box = await canvas.boundingBox();
  if (!box) {
    throw new Error('Canvas box unavailable.');
```

```

  }
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
  const orb = snapshot?.orbs.find((candidate) => candidate.active && candidate.tutorial && !candidate.captured);
  if (!orb) {
    throw new Error('Tutorial orb unavailable.');
```

```

  }
  const targetX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * 0.78;
  const targetY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * 0.78;
  const dx = orb.x - BLACK_HOLE_X;
  const dy = orb.y - BLACK_HOLE_Y;
  const length = Math.max(1, Math.hypot(dx, dy));
  const normalX = -dy / length;
  const normalY = dx / length;
  const start = worldToScreen(box, targetX - normalX * 150, targetY - normalY * 150);
  const mid = worldToScreen(box, targetX, targetY);
  const end = worldToScreen(box, targetX + normalX * 150, targetY + normalY * 150);
  return { start, mid, end };
}

function worldToScreen(box: { x: number; y: number; width: number; height: number }, x: number, y: number) {
  const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);
  const offsetX = (box.width - WORLD_WIDTH * scale) / 2;
  const offsetY = (box.height - WORLD_HEIGHT * scale) / 2;
  return {
    x: box.x + offsetX + x * scale,
    y: box.y + offsetY + y * scale
  };
}

```

tests/simulation.test.ts

```

import { describe, expect, it } from 'vitest';
import { BLACK_HOLE_X, BLACK_HOLE_Y } from '../src/game/constants';
import { quantizeGesturePath } from '../src/game/gestures';
import { Simulation, type TimedInput } from '../src/game/Simulation';
import { hashStringToUint, mulberry32 } from '../src/game/rng';

const options = {
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000
};

describe('deterministic simulation', () => {
  it('mulberry32 returns the same sequence for the same seed', () => {
    const a = mulberry32(hashStringToUint(options.seed));
    const b = mulberry32(hashStringToUint(options.seed));
    expect([a(), a(), a(), a()]).toEqual([b(), b(), b(), b()]);
  });

  it('same seed and same input timings produce identical replay outcomes', () => {
    const path = makeTetherCrossingPath(new Simulation(options));
    const inputs: TimedInput[] = [
      { kind: 'tap', t: 800, x: 520, y: 620 },
      { kind: 'swipe', t: 1600, x: path[0].x, y: path[0].y, x2: path[1].x, y2: path[1].y, path },
      { kind: 'tap', t: 3300, x: 540, y: 845 },
      { kind: 'swipe', t: 5100, x: 900, y: 1160, x2: 410, y2: 660 }
    ];
    const first = new Simulation(options).runInputs(inputs, 12000);
    const second = new Simulation(options).runInputs(inputs, 12000);
    expect(second).toEqual(first);
  });

  it('records tap and swipe events with simulation timestamps', () => {
    const sim = new Simulation(options);
    sim.step(1000);
    sim.applyTap(100, 200);
    sim.step(1000 / 60);
    sim.applySwipe(120, 300, 600, 500);
    const replay = sim.getReplayPayload();
    expect(replay.tapEvents).toHaveLength(1);
    expect(replay.swipeEvents).toHaveLength(1);
    expect(replay.tapEvents[0].t).toBe(1000);
    expect(replay.swipeEvents[0].x2).toBe(600);
    expect(replay.swipeEvents[0].path).toHaveLength(2);
  });

  it('a path crossing the readable tether captures the orb', () => {
    const sim = new Simulation(options);
    const path = makeTetherCrossingPath(sim);
    const before = sim.getSnapshot();
    const result = sim.applySwipePath(path);
    const after = sim.getSnapshot();
    expect(result.success).toBe(true);
    expect(result.target).toBe('tether');
    expect(result.orbId).toBeDefined();
    expect(result.path.length).toBeGreaterThanOrEqual(2);
    expect(after.score).toBeGreaterThan(before.score);
    expect(after.energyCaptured).toBeGreaterThan(before.energyCaptured);
  });
}

```



```

});

it('a path missing all tethers does not capture', () => {
  const sim = new Simulation(options);
  stepUntilFirstOrb(sim);
  const before = sim.getSnapshot();
  const result = sim.applySwipePath([
    { x: 80, y: 1760, t: 0 },
    { x: 180, y: 1840, t: 80 }
  ]);
  const after = sim.getSnapshot();
  expect(result.success).toBe(false);
  expect(result.target).toBe('empty');
  expect(result.message).toBe('MISS');
  expect(after.score).toBe(before.score);
  expect(after.energyCaptured).toBe(before.energyCaptured);
});

it('simplifies and quantizes long swipe paths for replay', () => {
  const points = Array.from({ length: 80 }, (_, index) => ({
    x: 120 + index * 9.25,
    y: 700 + Math.sin(index * 0.24) * 90,
    t: index * 7.5
  }));
  const path = quantizeGesturePath(points, 24);
  expect(path.length).toBeLessThanOrEqual(24);
  expect(path[0]).toEqual({ x: 120, y: 700, t: 0 });
  expect(path[path.length - 1].x).toBe(Math.round(points[path.length - 1].x));
});

it('backwards simple swipe calls still capture when crossing a tether', () => {
  const sim = new Simulation(options);
  const path = makeTetherCrossingPath(sim);
  const result = sim.applySwipe(path[0].x, path[0].y, path[1].x, path[1].y);
  expect(result.success).toBe(true);
  expect(result.target).toBe('tether');
});
});

function stepUntilFirstOrb(sim: Simulation) {
  for (let index = 0; index < 80; index += 1) {
    sim.step(1000 / 60);
    const orb = sim.getSnapshot().orbs.find((candidate) => candidate.active && !candidate.captured);
    if (orb) {
      return orb;
    }
  }
  throw new Error('No tutorial orb spawned.');
```

```

}

function makeTetherCrossingPath(sim: Simulation) {
  const orb = stepUntilFirstOrb(sim);
  const targetX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * 0.78;
  const targetY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * 0.78;
  const dx = orb.x - BLACK_HOLE_X;
  const dy = orb.y - BLACK_HOLE_Y;
  const length = Math.max(1, Math.hypot(dx, dy));
  const normalX = -dy / length;
  const normalY = dx / length;
  return [
    { x: targetX - normalX * 150, y: targetY - normalY * 150, t: 0 },
    { x: targetX + normalX * 150, y: targetY + normalY * 150, t: 96 }
  ];
}

```